

UNIVERSITY OF TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer Science

Final Dissertation

Design and Implementation of Linux Tracing Infrastructure

A Deep Dive from Traditional Frameworks to eBPF-Driven Observability

Supervisor
Bruno Crispo

Student
Giovanni Santini

Academic year 2025/2026

"I have no special talent. I am only passionately curious."

— Albert Einstein, letter to Carl Seelig, 11 March 1952

Contents

Abstract **1**

1 Introduction **3**
1.1 The tracing stack . . . 3
1.1.1 Event Sources . . . 3
1.1.2 Tracing Frameworks . . . 5
1.1.3 User Interfaces . . . 5
1.1.4 Frontends . . . 5

2 Tracing frameworks **6**
2.1 Ftrace . . . 6
2.1.1 The ftrace global ring buffer . . . 6
2.1.2 The tracer struct . . . 8
2.1.3 Static function tracing . . . 9
2.1.4 Dynamic function tracing . . . 10
2.1.5 The tracefs filesystem . . . 11
2.2 Perf . . . 11
2.2.1 The Perf ring buffer . . . 11
2.2.2 The Performance Monitoring Unit . . . 12
2.2.3 The perf_event_open() systemcall . . . 12
2.2.4 Security considerations . . . 13
2.3 Others . . . 13

3 Event Sources **14**
3.1 Tracepoints . . . 14
3.1.1 Tracepoint related types . . . 14
3.1.2 Tracepoint creation . . . 15
3.1.3 static_call and static_key mechanism . . . 16
3.2 Event Tracing . . . 17
3.2.1 Event classes . . . 18
3.3 Syscall Tracepoints . . . 18
3.4 Kprobes and Kretprobes . . . 19
3.4.1 Kprobe interface . . . 20
3.4.2 Kprobe implementation . . . 22
3.5 Perf events . . . 24

4 eBPF **28**
4.1 Introduction and overview . . . 28
4.2 Instruction Set . . . 28
4.2.1 Standardization . . . 29
4.2.2 Instructions and encoding . . . 29
4.2.3 Helper Functions . . . 30
4.2.4 Maps and platform variables . . . 30
4.2.5 Compiler Support . . . 30

4.3 The verifier . . . 30
4.3.1 Initial checks and preparations . . . 31
4.3.2 Register types . . . 32
4.3.3 Program simulation . . . 33
4.3.4 Range analsyis . . . 33
4.3.5 State pruning . . . 33
4.4 Program Types . . . 34
4.4.1 Attach points . . . 34
4.4.2 Trampolines . . . 34
4.4.3 BPF_PROG_TYPE_XDP . . . 35
4.4.4 BPF_PROG_TYPE_TRACEPOINT . . . 36
4.4.5 BPF_PROG_TYPE_LSM . . . 37
4.5 Helper functions and Kfuncs . . . 37
4.6 Maps . . . 39
4.7 BTF and CO-RE . . . 41
4.8 Security Considerations . . . 42
4.9 eBPF in production . . . 44

5 Kivebpf **45**
5.1 Introduction . . . 45
5.1.1 Resources . . . 45
5.1.2 Alert . . . 46
5.1.3 Components . . . 47
5.2 Detecting file accesses . . . 47
5.2.1 Uniquely identifying a file . . . 48
5.3 eBPF Program . . . 48
5.3.1 Implementation . . . 48

6 Conclusions **52**

Bibliography **52**

Abstract

Observability and deep kernel visibility are essential for maintaining the performance, reliability, and security of modern operating systems. Linux supports a vast number of powerful tracing capabilities to support this: over the history of the operating system, multiple frameworks and event sources have been introduced for different use cases. Due to the size and complexity of the Linux kernel, these capabilities are often poorly understood by developers. To demystify this landscape, an analysis of the Linux kernel codebase was conducted, gathering information through source code, git commits, mailing lists, technical articles, and conference talks. This work presents a comprehensive description of the tracing infrastructure provided by Linux kernel version 7.2, focusing on traditional frameworks like Ftrace and Perf, event sources like tracepoints and kprobes, and modern developments through eBPF. Finally, a real-world use case for eBPF in containerized environments is studied, introducing Kivebpf as an application that leverages modern Linux tracing infrastructure.

Use of Artificial Intelligence

Artificial intelligence tools, particularly Large Language Models (LLMs), were used to a limited extent during the preparation of this thesis. Their use was primarily intended to support information gathering, facilitate the understanding of complex code patterns, and improve the clarity and linguistic quality of selected portions of the text.

The content of this thesis is the author's own work. AI-generated suggestions were treated as supplementary material and critically reviewed by the author, who assumes full responsibility for the accuracy and integrity of the presented work.

1 Introduction

A *trace* is a record of various information about the execution of a program. Modern operating systems are responsible of several critical tasks, such as interacting with network and storage devices, to virtualise and sandbox untrusted processes, and to enforce security policies. For this reason, deep and dynamic kernel tracing has become a relevant concern for operating systems. In fact, tracing is useful for many use cases, including performance evaluation and optimization, security and policy enforcement, and observability, among others. Linux is a popular operating system widely used in data centers, high-performance computing and embedded systems; it supports a wide set of instrumentation capabilities for many use-cases, yet a comprehensive study of these is currently unavailable as documentation is often limited or outdated. The objective of this work is to understand the systems and interfaces implemented in the Linux kernel to support tracing of events, as of version 7.2 of the kernel released on 16 August 2026. To this end, a thorough analysis of the source code was conducted; additional information was gathered from commit messages, discussions in mailing lists and forums, and presentations in public conventions. Cross referencing this heterogeneous body of information was instrumental to develop an understanding of the reasons and technical merits of Linux's tracing infrastructure, and how it evolved over time.

Chapter 2 provides a description of classic Linux tracing framewors, focusing on the two most used ones: *ftrace* and *perf*. Chapter 3 describes the events that are supported on Linux. Chapter 4 focuses on eBPF. Chapter 5 introduces *Kevebpf* as a modern tracing application. Chapter 6 closes with the conclusions.

1.1 The tracing stack

An high level view of the tracing architecture supported by Linux is now presented. The architecture can be conceptually divided into a stack of multiple abstraction layers, where each abstraction interacts with the one over it and below it in both directions through a well-defined interface. Each conceptual layer contains several components which implement the logic for particular types of tracing. Figure 1.1 visually shows the architecture.

The layers and components are now described.

1.1.1 Event Sources

Event sources or simply *events* represent the specific information in a moment in time being traced. They may be triggered by hardware or software components, and may capture various types of information such as timestamps, register values, function arguments, etc. The lifetime of an event is usually managed by the tracing framework, such as registration, activation, deactivation and removal; the event uses one or more tracing framework to produce its side effect and output trace information.

Events are divided into *static* and *dynamic*. Static events are intentionally placed in the kernel code and are compiled into the kernel's binary, they include various forms of *tracepoints* and *perf events*. Dynamic events, instead, are created during runtime and may be placed in different parts of the kernel by the user, and example of dynamic events are *kprobes*. A brief descriptions of these types of events is now provided:

- *tracepoints*: static events placed in many paths of the kernel, they execute a callback function which traces arbitrary values depending on the particular tracepoint.
- *perf events*: performance event counters are hardware counters that are incremented when certain events happen, like wrong branch predictions or number of instructions fetched in a CPU. Perf counters may also be in software.

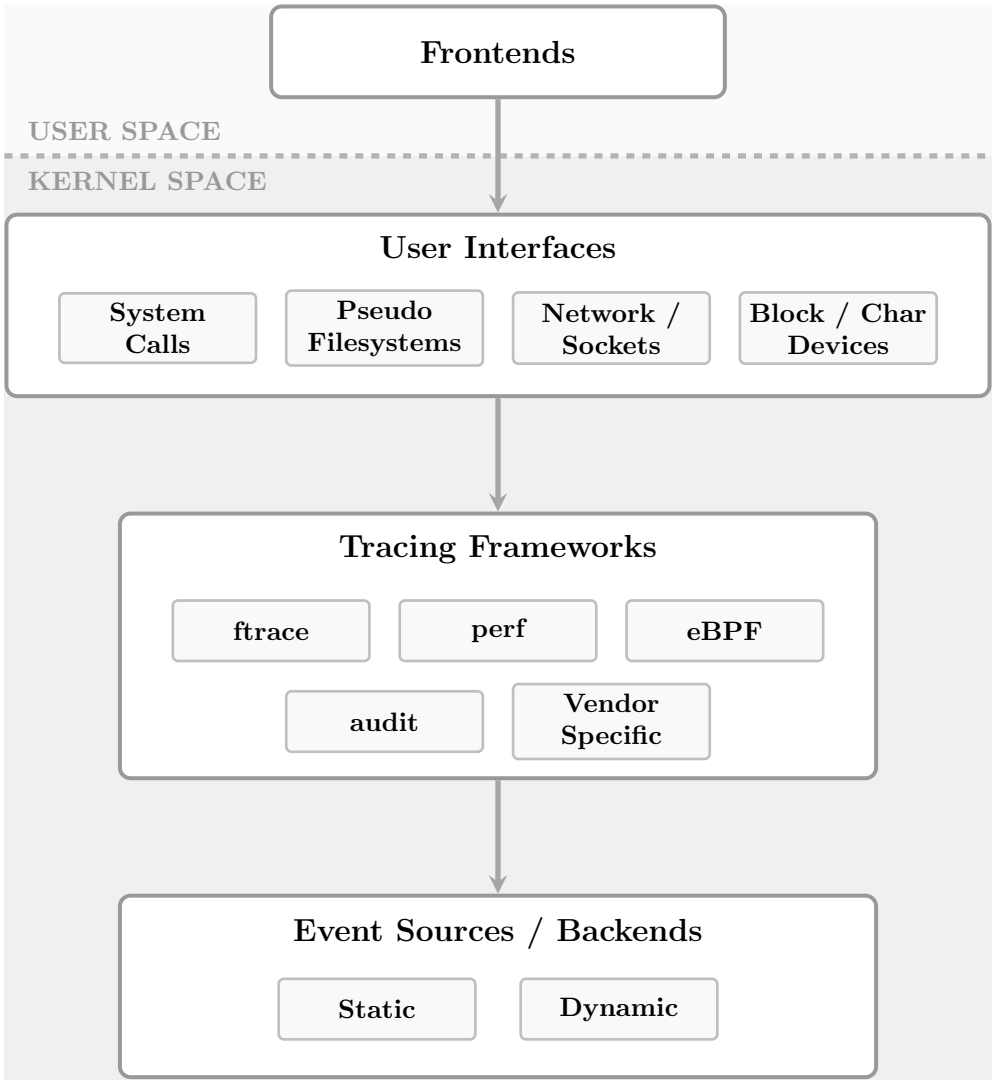


Figure 1.1: High-level architectural layout of the Linux tracing ecosystem.

- *kprobes*: dynamic functions patched in the kernel code at runtime which may trace arbitrary information.

1.1.2 Tracing Frameworks

Tracing frameworks are important architectural components. They manage events and their outputs, and can be accessed and managed via a set of userspace interfaces. Different frameworks provide alternative ways to interact with the events based on the application's needs. Notable frameworks that will be studied in this work are:

- *ftrace*: provides a single per-CPU global ring-buffer to hold all traces.
- *perf*: used originally for performance counters, provides per-CPU ring-buffers created at runtime via a systemcall and are memory mapped to user-memory for high-throughput communication.
- *eBPF*: mechanism to load and verify custom programs into the kernel, for tracing and observability reasons or to extend the functionalities of the kernel with custom logic.

1.1.3 User Interfaces

User interfaces bridge the boundary between kernelspace and userspace. They provide a stable interface for the user to interact and with a tracing framework to access its functionalities. The interface can be implemented as a *systemcall*, a dedicated *pseudo filesystem*, via the *network* such as though netlink, via *block* or *character* devices, though shared memory, or other means.

1.1.4 Frontends

Frontends are meant to provide a convenient and intuitive interface to the user, compared to the low-level kernel-to-user boundary. Since fontends are not developed inside the kernel, may solutions were developed by different groups and for different use-cases. An example list is provided below:

- *trace-cmd*: command line tool to interact with the ftrace tracer. It provides convenient commands such as "record" and "report" trace data, "list" to list the available events that can be recorded, "hist" to show an histogram of the events, and others.
- *bpftrace*: provides an high-level tracing language and runtime absed on eBPF. For example, the code `kprobe:do_nanosleep{printf("\%dsleeping\n",pid);}` creates a kprobe eBPF program on the `do_nanosleep` function which logs the `PID`.
- *systemtap*: provides a scripting language and instrumentation framework for monitoring and analyzing the behavior of running Linux systems. It allows users to define probes at kernel and user-space instrumentation points and execute custom handlers to collect data and investigate system behavior.
- *perf-tools*: a collection of Linux performance analysis tools built primarily on ftrace and perf events. It includes scripts for tracing system calls, examining kernel function execution, analyzing scheduling behavior, and investigating block I/O activity.
- *bcc*: the BPF Compiler Collection provides a framework for developing and running eBPF-based tracing and performance analysis tools. It combines kernel instrumentation with user-space programs, typically written in Python or C++, and includes utilities for analyzing processes, system calls, networking, and other kernel activities.
- *strace*: a system-call tracing utility that monitors the system calls and signals of a process. It can report system-call arguments, return values, errors, and execution durations, making it useful for debugging applications and investigating their interactions with the kernel.
- *ltrace*: a library-call tracing utility that monitors calls made by a process to dynamically linked libraries. It can display function arguments and return values, helping to investigate application behavior and interactions with shared libraries.

2 Tracing frameworks

A *tracing framework* is a set of APIs and behaviours designed to interface event sources with other kernel objects and user front-ends. By decoupling the event sources from the serialization and consumption pipelines, tracing frameworks provide an architectural foundation for kernel observability. In general, a tracing framework should support the registration and de-registration of event sources, whether static or dynamic, enabling and disabling, and producing or consuming their side effects. Usually, user-space interacts with the functionalities exposed by a tracing framework through a well-defined stable user-space API, such as through a filesystem or a dedicated system call. Notably, there is no single, unified tracing framework on Linux for registering and accessing all kinds of events, but there are multiple solutions tailored to different use cases. The purpose of this chapter is to provide an overview of the design and implementation of the two major tracing frameworks present in Linux: *ftrace* and *perf*.

2.1 Ftrace

Ftrace is a versatile tracing framework that supports registration, deregistration, reading, writing and other operations on various types of trace events. It defines a *tracer* abstraction to manage a set of events in groups, and a *global ftrace ring buffer* to produce and consume all the event data. Additionally, it implements functionalities used by many types of tracers and events, such as the registration of *hooks* (altering a funciton's execution to jump to a callback) on most kernel functions and other code patching routines. This chapter explores the aforementioned subjects in more detail.

Ftrace was originally implemented in the Linux 2.6 kernel to support *function tracing*, that is, capturing a funciton's execution data such as its arguments and name whenever the function is entered or exited, in real time. Over time, the same infrastructure was reused for other use cases and tracers, consequently the name "ftrace" does not retain any of its original meaning. For this reason, the term "function tracer" will be used to refer to the specific tracer used for functions, and "ftrace" for the wider infrastructure.

Modern ftrace is most useful for tracing global events that happen on any CPU and by any process, although it can be configured to filter events with specific conditions, enabling ftrace to be adapted to various use-cases.

2.1.1 The ftrace global ring buffer

A core component of ftrace is its *global ring buffer*, represented at its highest abstraction level by the `struct trace_array` type. It provides an interface to write data and consume it, supporting both a producer-consumer mode and an overwrite mode. The two modes are distinguished by how writers behave when the buffer is full: in producer-consumer mode the producer fails to write, otherwise it overwrites the data.

Since events are expected to happen at a high frequency and from various different contexts (such as process, interrupt, etc), the design of the ring buffer needs to meet specific requirements. These requirements include:

- Writers must handle reentrancy safely, ensuring successful execution even when interrupted by hardware interrupts or non-maskable interrupts (NMIs).
- The design must support high concurrency for multiple writers and readers distributed across multiple CPU cores.
- Synchronization overhead must be minimized to efficiently handle a high volume of concurrent writes from high-frequency events.

The solution implemented in Linux in the file `$LINUX/linux/kernel/trace/ring_buffer.c` fulfills all these requirements using a lockless design for both reader-to-writer and writer-to-writer interactions. Some key design decisions are briefly discussed, although a detailed description is provided in the kernel's documentation.

Per-CPU ring-buffers

Each CPU uses its own per-CPU ringbuffer composed of a doubly-linked list of pages; consequently, writers on different CPUs do not need to synchronize among each other. In order to read a full trace in chronological order, a reader iterates over all the per-CPU buffers using time information to determine the order of the events. Another key detail is to store the reader page externally to the ring buffer linked list: writers never interact with the reader page and readers swap the reader page via atomic operations so no lock is used on the same per-CPU ring buffer.

Reserve and commit pattern

To support preemption, writers use a *reserve and commit* pattern which defines three phases to perform a write: first the writer reserves allocation space in a per-CPU ring buffer, then it writes with exclusive access to that region, and finally commits the region. The commit will be in a pending state until all regions before it are also committed; therefore when a writer reserves a region but gets preempted by another writer, the new writer will not have access to the previously reserved region and its changes will not get registered until the first writer completes. This mechanism ensures that readers only process fully committed entries.

In C code, the commit pattern looks like the following function from `$LINUX/linux/kernel/trace/trace_uprobe.c`, commented and simplified for readability:

```
1  static void __uprobe_trace_func(struct trace_uprobe *tu,
2          unsigned long func, struct pt_regs *regs,
3          struct uprobe_cpu_buffer *ucb,
4          struct trace_event_file *trace_file)
5  {
6    struct trace_event_buffer fbuffer;
7    void *data;
8    int size = ...;
9
10   /* First, reserve an area */
11   entry = trace_event_buffer_reserve(&fbuffer, trace_file, size);
12   if (!entry)
13     return;
14
15   data = /* get the data pointer from and entry */
16
17   /* Second, copy the data into the ring buffer page */
18   memcpy(data, ucb->buf, ucb->dsize);
19
20   /* Third, commit */
21   trace_event_buffer_commit(&fbuffer);
22 }
```

The relevant interactions with the ring buffer are: the reservation on line 11, the writing on line 18 via a `memcpy()` and the commit on line 21.

Trace sequences

Because printf-like string formatting is an expensive operation, an event is stored in the ring buffer in binary form without any formatting. When a consumer reads the ring buffer though the user-space API, the relevant contents of the ring buffer are formatted with a printf-style function, `trace_seq_vprintf()`, into a trace sequence of type `struct trace_seq`. A trace

sequence stores a pre-allocated buffer of 8K bytes that can be written to and read from. After formatting, the data is copied from the trace sequence to user memory. Moving the formatting procedure to the reader improves the writer's latency which is critical for high frequency events.

2.1.2 The tracer struct

An important abstraction in ftrace is the *tracer*, implemented as the `struct tracer` type. This object provides an interface to enable or disable a set of trace events, with custom formatting if necessary. Any tracer implements a set of functions to be invoked during its lifetime, such as initialization, starting, stopping, reading, etc. An use case for a tracer would be, for example, to trace all functions executed, or to trace all context switches. The user interacts with the tracer via the *tracefs* filesystem API which will bee discussed later. Many tracers are available, but only one tracer can be enabled at a time.

The default tracer is the *nop* tracer which is a "dummy" tracer that does not enable any event. The instantiation of such tracer looks like this:

```
1  /* Supported output options */
2  enum {
3    TRACE_NOP_OPT_ACCEPT = 0x1,
4    TRACE_NOP_OPT_REFUSE = 0x2
5  };
6
7  /* Options for the tracer */
8  static struct tracer_opt nop_opts[] = {
9    { TRACER_OPT(test_nop_accept, TRACE_NOP_OPT_ACCEPT) },
10   { TRACER_OPT(test_nop_refuse, TRACE_NOP_OPT_REFUSE) },
11   { } /* Always set a last empty entry */
12 };
13
14 static struct tracer_flags nop_flags = {
15   .val = 0, /* By default: all flags disabled */
16   .opts = nop_opts
17 };
18
19 struct tracer nop_trace __read_mostly =
20 {
21   .name = "nop",
22   .init = nop_trace_init,  /* dummy functions, do not do anything */
23   .reset = nop_trace_reset,
24   .flags = &nop_flags,
25   .set_flag = nop_set_flag,
26   .allow_instances = true,
27 };
```

The `nop_trace` tracer is instantiated in line 19, where the `init` and `reset` members on line 22 and 23 respectively are pointers to functions that are supposed to bring up and remove all events managed by the tracer, but, in this implementation, do nothing. The tracer is registered to the ftrace infrastructure via the `register_tracer()` function which is called during kernel startup.

The following is a non exhaustive list of available tracers:

- *function*: trace all kernel functions when entered.
- *function_graph*: trace both function entry and exit. It can be used to draw a graph of function calls.

- *blk*: block tracer, for detailed IO tracing.
- *hwlat*: hardware latency tracer.
- *irqsoff*: trace the areas that disable interrupts and saves the trace with the longest maximum latency.
- *preemptoff*: similar to *irqsoff* but traces and records the amount of time for which preemption is disabled.
- *wakeup*: traces and records the maximum latency that it takes for the highest priority task to get scheduled after it has been woken up.

2.1.3 Static function tracing

To support tracing function enter and exit events, ftrace implements a system to instrument any function in order to alter its execution and trace the event. For clarity, the following section will use the term *trace* function or *tracing* function to refer to the function that performs the writing of the event onto the ftrace ringbuffer, and the *traced* function to the function that is being instrumented.

If function tracing is enabled, whenever a C function is executed, such that the program counter reaches the first instruction belonging the function, then the execution should jump to a tracing function and return to the traced function without altering the registers. Likewise, when the program counter reaches the end of the function to return, it should jump to a tracing function and later jump to the original return address. This behaviour should be supported transparently, without altering existing code or to expect developers to statically place instrumentation onto any function.

The aforementioned behaviour can be implemented via special compiler support introduced for this specific use case. When *gcc* is used with the `-pg` command line flags, a jump to a trace function called `mcount()` will be generated after the prologue of any function on the ARM CPU architecture, or a jump to `__fentry__()` before the prologue on the x86-64 architecture. For function exits, the trace function will change the return address of the traced function with the address of a trace function. This return address replacement happens on x86 by changing the return address of a function from the stack, where it is pushed in the function's prologue as per C calling conventions, and on the ARM architecture by changing the register olding the return value (R14 for 32 bit, or x30 on 64 bit ARM).

This technique referred to as "static function tracing" because the compiler instruments each function permanently at compile time, and can be enabled via the `CONFIG_FUNCTION_TRACER` kernel compile option.

For example, the function `sum()` reported below on the left column is compiled by gcc 16.1 into x86-64 assembly with `-O1` flag as shown on the right column:

```
int sum(int a, int b) {
    return a + b;
}
```

```
"sum(int, int)":
        lea     eax, [rdi+rsi]
        ret
```

The same compiler would compile the same function as follows, this time with the `-pg` flag added:

```
int sum(int a, int b) {
    return a + b;
}
```

```
"sum(int, int)":
        call    __fentry__
        lea     eax, [rdi+rsi]
        ret
```

In the second version, the compiler added a call to `__fentry__()` before performing the sum operation. The `__fentry__()` function is a regular function that can be implemented arbitrarily , which in the case of Linux would be used for tracing purposes.

A shortcoming of this implementation is that every kernel function suffers the time penalty of an additional function call at minimum, causing some performance loss in case tracing is compiled in (`__fentry__()` calls are present) but not turned on (nothing is being traced). This issue is solved with "dynamic function tracing", which will be discussed next.

2.1.4 Dynamic function tracing

In order to not have any performance penalty when function tracing is supported but not turned on, ftrace provides a way to dynamically patch functions to add or remove tracing instrumentation. Dynamic tracing is enabled with the kernel compile option `DYNAMIC_FTRACE`.

When dynamic function tracing is enabled, the kernel will patch all calls to `mcount()` or `__fentry__()` with `NOP` operations at boot-time, which do not perform any operation. When tracing is enabled, the `NOP` instructions are patched with a call to `ftrace_caller()` or `ftrace_regs_caller()` which can access a copy of all original registers from the traced function. The information of all the locations of all `mcount()` calls is generated at compile time via the `recordmcount` program which is invoked by the build infrastructure. The tool stores all the locations of these functions in the `__mcount_loc` ELF section, which is linked in the original object file. Through this information, ftrace know what and where to patch code. This system allows for minimum overhead in the case where tracing is disabled.

The tracing function `ftrace_caller` is also referred to as a "trampoline" because its instructions are dynamically patched to jump to the functions which are currently enabled depending on the current tracer o particular events, represented in the `struct trace_ops`. Ftrace provides the function `register_ftrace_function()` to register a trampoline which is implemented by an architecture dependent function, `create_trampoline()`.

For example, consider the following assembly function for x86:

```
some_kernel_func:
  PUSH RBP
  MOV RSP, RBP
  ...
  RET
```

When tracing is disabled but dynamic tracing is supported, some `NOP` operations would be patched in at the start like in the next example:

```
# Symbol where instructions like: CALL some_kernel_func will jump to.
some_kernel_func:
  NOP
  NOP
# actual start of the function
  PUSH RBP
  MOV RSP, RBP
  ...
  RET
```

When tracing is enabled, these same `NOP` functions are replaced with a call to `ftrace_caller()`:

```
# Symbol where instructions like: CALL some_kernel_func will jump to.
some_kernel_func:
  CALL ftrace_caller
  NOP
# actual start of the function
  PUSH RBP
  MOV RSP, RBP
  [...]
  RET
```

2.1.5 The tracefs filesystem

The ftrace subsystem needs to provide an interface for the user to interact with it. This is done through the *tracefs* filesystem. Compared to other user interfaces like systemcalls and ioctls, many Linux kernel developers prefer custom filesystems or custom entries in an already existing filesystem as a method to provide new functionalities to the user. This is the case because filesystems make the interface explicit and visible in user space; they also make it easier to write scripts which perform administrative functions. Originally, ftrace used the *debufgs* filesystem but it grew enough to motivate the addition of the current interface, tracefs. In this section, the term *$TRACEFS* will be used to represent the location where the tracefs filesystem is mounted.

Users can access the formatted content of all ring buffers by reading the `$TRACEFS/trace` file without consuming the buffer, or `$TRACEFS/trace_pipe` to consume it. Additionally, they can get information about the buffer state from several files beginning with `buffer_*`. To access a particular per-CPU buffer, they can use the files from the directory `$TRACEFS/per_cpu/$CPU_NUMBER/`. Users can query a list of the tracers supported by the running kernel by reading the file `$TRACEFS/available_tracers`. They can select a tracer by writing its name to the file `$TRAEFS/current_tracer` and tune its options from `$TRACEFS/options/$OPTION_NAME`. Additionally, tracefs provides a way to specify filters for events, so that only certain evens in certain contexts will be logged.

A more thorough explanation of the tracefs interface is available in the kernel documentation.

2.2 Perf

The *perf* subsystem is another tracing framework provided by the kernel to manage and interact with trace events. It was originally designed as an interface to read hardware performance counters (hence the name "perf" as in "performance"), which are increasing integers exposed by the hardware that store various hardware-related information such as number of cache misses or number of incorrect branch predictions in the case of a CPU. Counters are particularly valuable for performance analysis, as they offer low-level visibility into system behavior. Conceptually, they are straightforward as they record the occurrence and frequency of specific events. However, the perf subsystem has been extended to support other event types, such as static and dynamic tracepoints, that log richer contextual information, offering an alternative interface to ftrace. Event counters provided through perf are referred to as *perf events*, and are broadly categorized as either *software* or *hardware* events depending on the underlying source that triggers the event or increments the counter.

2.2.1 The Perf ring buffer

To produce and consume event data, the perf infrastructure implements its own ring buffer data structure, with some similarities and key differences compared to ftrace's implementation. This is used for *sampling* events that periodically write measurements to the buffer, as opposed to *counting* events which count the aggregate number of events that occur and commonly do not use the ring buffer. Unlike ftrace, the perf ring buffer is not a global static ring buffer, but it is rather created by the user via a systemcall and connected to the lifetime of another object, such as a file descriptor. Another key difference from ftrace is the fact that the perf ring buffer is a contiguous area of virtual memory that can be memory mapped via `mmap()` to the user's virtual address space, while ftrace uses a linked list of pages which are read through a filesystem. Memory mapping the ring buffer removes the need of repeated `read()` systemcalls which add overhead. The memory mapped area follows the structure of `struct perf_event_mmap_page`, containing a pointer to the *head* producer, which is updated by the kernel, and *tail* consumer, updated by the user program. It is the responsibility of the user to atomically update the tail pointer after consuming the data, while the kernel updates the head pointer as it fills the buffer. Similarly to ftrace, the perf ring buffer uses a reserve and commit pattern with the `perf_output_begin()` and `perf_output_end()` functions.

For hardware events, perf uses architecture-specific *auxiliary (AUX) ring buffers*. Since they are implemented based on a particular architecture's needs, they provide lower latency for high

frequency events compared to a generic implementation.

2.2.2 The Performance Monitoring Unit

To manage perf events, the perf subsystem introduces the *Performance Monitoring Unit (PMU)* abstraction represented with the `struct pmu` type. This object manages the entire life cycle of an event: from registering and unregistering it, to starting and stopping tracing, and reading the counter.

All the PMUs are registered with the `perf_pmu_register()` function and added to a global list of all PMUs, called "`pmus`".

```
int perf_pmu_register(struct pmu *pmu, const char *name, int type);
```

This function takes a `struct pmu`, a name and an integer variable named "type"; the name is used to register a directory in the *sysfs* filesystem under `$SYSFS/bus/event_source/devices/$NAME/`, where `$SYSFS` is the location of the sysfs filesystem, and `$NAME` is the name of the PMU passed to the registration function. Through the sysfs API the user can enumerate and get useful metadata about the events, but reading the actual event must be done through a systemcall as described in the next section, or through a memory mapped region. The "type" argument will specify which binary format the event will use.

Each PMU contains pointer to functions to manage one or more events; the structure looks like the following:

```
struct pmu {
  // ...
  void (*pmu_enable) (struct pmu *pmu); /* optional */
  void (*pmu_disable) (struct pmu *pmu); /* optional */
  int  (*event_init) (struct perf_event *event);
  int  (*add) (struct perf_event *event, int flags);
  void (*del) (struct perf_event *event, int flags);
  void (*start) (struct perf_event *event, int flags);
  void (*stop) (struct perf_event *event, int flags);
  // ...
};
```

A more detailed look at how an event counter is registered will be taken in a later chapter.

2.2.3 The perf_event_open() systemcall

The user mainly interacts with the perf subsystem through a custom systemcall: `perf_event_open()`. Via this API, the user can access event counters or the ring buffer, specifying the events to be traced and other information such as the context (for example, per per-thread or per-CPU). This systemcall does not have a *libc* wrapper.

The function signature of the systemcall is the following:

```
int perf_event_open(struct perf_event_attr *attr,
                    pid_t pid, int cpu, int group_fd,
                    unsigned long flags);
```

The `struct perf_event_attr` type is used to specify which event should be monitored and some variables such as the frequency of sampling or whether to exclude counting on certain contexts such as user or kernel. The full definition of the structure is described in the man page. The "pid" and "cpu" arguments are used to specify which process and CPU to monitor, the allowed values and their meanings are also explained in the manual page. Finally, the "group_fd" argument can be used to create events in groups which is useful to measure multiple events simultaneously, and the "flags" argument specify additional options.

The return value is a new file descriptor or -1 on error. The user can use the file descriptor to read the counter, or to memory map the ring buffer to userspace to access its memory directly.

Calls to `ioctl()` using the file descriptor can be issued to enable or disable the event. Multiple counters can be kept open at a time, and the counters can be polled with `poll()`.

2.2.4 Security considerations

Performance counters introducessubstantial security risks regarding the potential leakage of sensitive data provided by monitored events. These events can leak information such as hardware configuration, content of registers, execution data (context switches, pages faults) etc, with various degrees of risk. Therefore, the perf subsystem supports several security frameworks: the Unix privilege model, POSIX capabilities and Linux security modules (LSM).

In particular, usage of the systemcall is restricted to privileged users (Unix `root`) while other users must have the `CAP_PERFMON` capability. The systemcall also registers a LSM hook which can be used by any security module to provide custom access policies. Additionally, the subsystem register the `perf_event_paranoid` parameter, accessible in `$PROCFS/sys/kernel/perf_event_paranoid`, which can be set to provide stricter or looser access checks, as described in the systemcall manual page.

2.3 Others

There are other minor tracing frameworks for specific use-cases or maintained by vendors for their own hardware, both in and out of the kernel source tree. Some of these frameworks are reported below:

- *audit*: a framework that provides records about security-relevant events, logging other informatinon such as timestamps, PID, etc. The user can specify some rules that, if matched, trigger a security event. The audit framework provides several hooks in various parts of the kernel, such as when entering a systemcall, when an inode is modified, when a task is created, etc. It communicates to userspace via netlink.

- *syslog*: ring buffer used for debug messages and logging. Data is produced via `printk()` and can be consumed in userspace from `$PROCFS/kmsg`.

- *coresight*: tracing framework for ARM-based SoC, managing hardware-specific "sources", "links" and "sinks". It integrates also with perf.

- *MIPI system trace module (STM)*: tracing framework based on MIPI specifications that allows kernel drivers and userspace to output software trace messages directly into hardware trace modules via character devices, routing data through physical debug pins or hardware sinks.

- *relayfs*: transport infrastructure that provides efficient per-CPU memory-mapped ring buffers to transfer large volumes of trace data from kernel space to userspace, bypassing traditional filesystem overhead.

- *blktrace*: specialized tracing framework for the block layer that hooks directly into request queues to capture the complete lifecycle of I/O requests from creation to completion, exporting event data to userspace via relay channels.

3 Event Sources

The previous chapter explained various frameworks for supporting event tracing. This chapter discusses some of the most common types of events used in Linux, focusing on their registration, implementation and interaction with the aforementioned frameworks. In general, an event is triggered when internal or external conditions are met, causing the system to execute logic that reacts to the occurrence. The lifetime of an event is managed by a tracing framework, and the event may use one or more frameworks to produce the tracing data. A distinction is made between *static* events and *dynamic* events: static events are placed by kernel developers in predetermined locations and statically compiled in the kernel, while dynamic events are registered during runtime and can be registered in non-predetermined locations.

3.1 Tracepoints

A *tracepoint* is a system that calls an arbitrary function when triggered, which is referred to as a "callback". Trigger points are statically placed all over the kernel's codebase between regular code; there are more than 1000 tracepoints available in Linux 7.2. A tracepoint can be either set as "on", meaning a callback is connected to it, or "off" meaning no callback is present. The callback can accept custom parameters for tracing purposes. The usage of tracepoints is conceptually simple, which will be explained with several examples.

Consider the following code:

```
1 void foo(void) {
2   int x = bar();
3   /* ... */
4   trace_foo(x); /* Tracepoint trigger placed here */
5   /* ... */
6   return baz(x);
7 }
```

Here the function `foo()` calls at some point `trace_foo()` on line 4 with the variable `x` passed as an argument. `trace_foo()` is a function generated by the tracpoint API for a particular tracepoint called "foo", along side with other functions. In this case, if the tracepoint is currently active, then this function will jump to callback function specified when registering the tracepoint. The callback can use the `x` variable for any purposes, such as to provide it to the user by writing it to the ftrace or the perf ring buffer.

3.1.1 Tracepoint related types

The key types that the tracepoint API defines are `struct tracpoint` which contains all information related to the tracepoint, `struct tracepoint_func` containing the callback, and `structtracepoint_ext` specifying extended attributes. They are reported below from the file `$LINUX/include/linux/tracepoint-defs.h:26`, where `$LINUX` represents the root of the Linux kernel source code:

```
1 struct tracepoint_func {
2   void *func;
3   void *data; int prio;
4 };
5
6 struct tracepoint_ext {
```

```
7   int (*regfunc)(void);
8   void (*unregfunc)(void);
9   /* Flags. */
10  unsigned int faultable:1;
11 };
12
13 struct tracepoint {
14   const char *name;    /* Tracepoint name */
15   struct static_key_false key;
16   struct static_call_key *static_call_key;
17   void *static_call_tramp;
18   void *iterator;
19   void *probestub;
20   struct tracepoint_func __rcu *funcs;
21   struct tracepoint_ext *ext;
22 };
```

The most relevant members of `struct tracepoint` are its "name" on line 14, which is used by the user to identify a particular tracepoint, and the "funcs" member on line 20 used to specify one or more functions to be called when the tracepoint is executed (which are of type `struct tracepoint_func`). The "ext" (extended) member on line 21 contains optional functions to be called when registering or unregistering the tracepoint. Additionally, the static-key and static-call related fields are performance optimizations which will be briefly explained later in this section.

3.1.2 Tracepoint creation

To create a tracepoint, the macro `DECLARE_TRACE()` should be used. This macro takes three arguments: the name of the tracepoint, the prototype of the callback, and the name of the arguments what should be passed to the callback (relevant to the specific scope where the tracepoint is placed). When expanded, it sets up the necessary static storage for the tracepoint and generates a set of functions to interact with it.

For example, consider the following macro statement:

```
1 DECLARE_TRACE(foo_tp,
2   TP_PROTO(bool is_bar),
3   TP_ARGS(bar));
```

This declares a tracepoint called `foo_tp` which calls a function that looks like `foo_functi on(boolis_bar)`. This function will be called with the `bar` argument, like `foo_function(bar)`.

Consider now a real example from `$LINUX/include/trace/events/sched.h`:

```
1  #undef TRACE_SYSTEM
2  #define TRACE_SYSTEM sched
3
4  #if !defined(_TRACE_SCHED_H) || defined(TRACE_HEADER_MULTI_READ)
5  #define _TRACE_SCHED_H
6
7  #include <linux/tracepoint.h>
8
9  DECLARE_TRACE(sched_entry,
10   TP_PROTO(bool preempt),
11   TP_ARGS(preempt));
12
```

```
13 #endif /* _TRACE_SCHED_H */
14
15 /* This part must be outside protection */
16 #include <trace/define_trace.h>
```

In the first line, the `TRACE_SYSTEM` symbol specifies the subdirectory of tracefs where this tracepoint will be registered for access to the user. The code follows by including the file `$LINUX/tracepoint.h` in line 7 which contains the `DECLARE_TRACE()` macro definition, and proceeds to declare the `sched_entry` tracepoint on line 9. It finally includes `trace/define\_trace.h` on line 16 which defines the various functions for the tracepoint and static storage, such as functions for registration, de-registration, calling the callback, and a function to check if the tracepoint is active, among others. After all macro expansions, the following functions are generated and can be used throughout the codebase:

- `register_trace_sched_entry()`: takes the callback and sets it up to be called by the tracepoint.
- `unregister_trace_sched_entry()`: unregisters the tracepoint.
- `trace_call__sched_entry()`: calls the callback.
- `trace_sched_entry()`: calls the callback if the tracepoint is. enabled.
- `trace_sched_entry_enabled()`: checks if the tracepoint is enabled.

It should be noted that the `define_trace.h` header needs to be placed outside any inclusion guards so that it will be always included by the preprocessor. This is used for a trick with the preprocessor where the `define_trace.h` header includes the file that is including it, therefore including itself again and redefining the macro `DECLARE_TRACE()`, but using `#define` guards to change its definition. For example, if file "X" includes `define_trace.h`, then it will include itself, "X", which will include `define_trace.h` another time, and so on; this cycle may repeat multiple times, until some condition is met. The reason is to redefine the `DECLARE_TRACE()` macro in different ways to generate different code. This is also used in other parts of the kernel, namely "event tracing" which will be explained later in this section.

3.1.3 static_call and static_key mechanism

After declaring a tracepoint with `DECLARE_TRACE()`, the function `trace_sched_entry()` can be used to trigger the tracepoint. The callstack looks like the following, where the function on top calls the function underneath:

```
trace_sched_entry(...)
  __do_trace_sched_entry(...)
      static_call(tp_func_sched_entry)
```

The *static call* on the last line is a mechanism that uses code patching to replace global function pointers. It allows directs calls to be used instead of indirect calls. In this particular case, instead of having a pointer to the callback function stored somewhere in the tracepoint structure which needs to be loaded from memory, a call to its address is directly patched where the `static_call()` call lives therefore removing one layer of indirection. There may be multiple callbacks registered, in this case the call will be patched with an iterator which simply loops over all registered callbacks and calls them.

Before calling `static_call()`, a check is made to determine if the tracepoint is active or not by checking if the callback address is present. To avoid a conditional branch instruction in the if statement, the code is patched at runtime via a mechanism called `static_key` that replaces the if statement with a jump or a NOP instruction depending on whether the event is active or not. Note that this is different than `static_call()` because it replaces a conditional branch

(compare + jump instructions) and not a function call. This decreases the runtime performance impact of a disabled tracepoint, as the only effective penalty is overall increased code section size due to the tracepoint generated functions.

3.2 Event Tracing

A tracepoint provides a solution to call a callback function when triggered. Additional work is necessary to make this system interact with the rest of the tracing frameworks in the kernel, such as ftrace and perf. To simplify this, the macro `TRACE_EVENT()` should be used instead of `DEFINE_TRACE()`. Creating a tracepoint using this macro is referred to as "event tracing", which is not to be confused with "perf events".

The `DEFINE_TRACE()` macro is responsible for creating a proper tracepoint, creating a callback that can be hooked to the tracepoint recording some formatted data into the tracer ring buffer, supporting both perf and ftrace. To accomplish this, the `TRACE_EVENT()` macro is broken into six components, which correspond to the parameters of the macro:

- *name*: the name of the tracepoint to be created
- *prototype*: the prototype for the tracepoint callbacks
- *args*: the arguments that match the prototype
- *struct*: the structure that a tracer could use (but is not required to) to store the data passed into the tracepoint
- *assign*: code written in C to assign the data to the structure.
- *print*: code written in C to output the structure in human readable ASCII format.

For example, consider the trace event in `$LINUX/fs/fuse/fuse_trace.h:126`:

```
1  TRACE_EVENT(fuse_request_end,
2    TP_PROTO(const struct fuse_req *req),
3
4    TP_ARGS(req),
5
6    TP_STRUCT__entry(
7      __field(dev_t,    connection)
8      __field(uint64_t,  unique)
9      __field(uint32_t,  len)
10     __field(int32_t,  error)
11   ),
12
13   TP_fast_assign(
14     __entry->connection  =  req->chan->conn->dev;
15     __entry->unique    =  req->in.h.unique;
16     __entry->len    =  req->out.h.len;
17     __entry->error    =  req->out.h.error;
18   ),
19
20   TP_printk("connection %u req %llu len %u error %d", __entry->connection,
21       __entry->unique, __entry->len, __entry->error)
22 );
```

This code creates the trace event `trace_fuse_request_end` using the specified parameters to format the event. The expansion of this macro uses the same trick as `DEFINE_TRACE()` by

redefining itself a total of 7 times which will not be covered here due to the high complexity of the expansion. After the expansion, the function `trace_fuse_request_end()` can be called anywhere in the code to trigger the tracepoint, which will use ftrace to produce its output, formatted as specified in `TP_printk()` on line 20 using the variables from `TP_fast_assign()` on line 13. Additionally, the `TRACE_EVENT()` macro registers entries in tracefs to disable and enable the event (`$TRACEFS/events/fuse/fuse_request_end/enable`).

3.2.1 Event classes

Since `TRACE_EVENT()` generates a lot of code and increases the size of the kernel binary, the macro `DECLARE_EVENT_CLASS()` and `DEFINE_EVENT()` should be used for tracepoints that share some code. The class defines the set of variables and formatting which can be reused by multiple events belonging to that class through `DEFINE_EVENT()`.

3.3 Syscall Tracepoints

For various use cases, it may be necessary to trace accesses to systemcalls. For this reason, special tracepoints are placed specifically for systemcalls which are triggered when the function is entered and exited. They are used to trace both systemcall name and its arguments to a tracing framework, for any systemcall. An explaination of the implementation is now provided.

A systemcall is defined by the `SYSCALL_DEFINEX()` family of macros. Taking `SYSCALL_DEFINE0()` as an example, the macro expands to the following code on x86, extracted from `$LINUX/arch/x86/include/asm/syscall_wrapper.h:244`:

```
1 #define SYSCALL_DEFINE0(sname)            \
2   SYSCALL_METADATA(_##sname, 0);          \
3   static long __do_sys_##sname(const struct pt_regs *__unused);  \
4   __X64_SYS_STUB0(sname)            \
5   __IA32_SYS_STUB0(sname)            \
6   static long __do_sys_##sname(const struct pt_regs *__unused)
```

Notice on line 2 the macro `SYSCALL_METADATA`, which expands to the following code, extracted from `\$LINUX/include/linux/syscalls.h:178`:

```
1  #define SYSCALL_METADATA(sname, nb, ...)      \
2    static const char *types_##sname[] = {      \
3      __MAP(nb,__SC_STR_TDECL,__VA_ARGS__)    \
4    };              \
5    static const char *args_##sname[] = {      \
6      __MAP(nb,__SC_STR_ADECL,__VA_ARGS__)    \
7    };              \
8    SYSCALL_TRACE_ENTER_EVENT(sname);      \
9    SYSCALL_TRACE_EXIT_EVENT(sname);      \
10   static struct syscall_metadata __used      \
11     __syscall_meta_##sname = {        \
12     .name     = "sys"#sname,      \
13     .syscall_nr  = -1,  /* Filled in at boot */  \
14     .nb_args   = nb,        \
15     .types    = nb ? types_##sname : NULL,  \
16     .args    = nb ? args_##sname : NULL,  \
17     .enter_event  = &event_enter_##sname,    \
18     .exit_event  = &event_exit_##sname,    \
19     .enter_fields  = LIST_HEAD_INIT(__syscall_meta_##sname.enter_fields), \
20   };              \
21   static struct syscall_metadata __used      \
```

```
22      __section("__syscalls_metadata")      \
23     *__p_syscall_meta_##sname = &__syscall_meta_##sname;
```

On line 8 and 9 the macros `SYSCALL_TRACE_ENTER_EVENT()` and `SYSCALL_TRACE_EXIT_EVENT()` are responsible for creating the tracepoint, and on line 10 a `struct syscall_metadata` is created which is used to save some information about the syscall and its arguments. Finally on line 21 a pointer to the metadata struct is created in the section `__syscalls_metadata` which contains all metadata from all systemcalls.

Whenever a systemcall is executed by the user, the following functions are called from top to bottom:

```
entry_SYSCALL_64
   do_syscall_64
      syscall_enter_from_user_mode
         syscall_trace_enter
            trace_syscall_enter
              trace_sys_enter
```

The last function, `trace_sys_enter`, is a trace event defined in `$LINUX/include/trace/events/syscalls.h`:

```
1  TRACE_EVENT_SYSCALL(sys_enter,
2
3    TP_PROTO(struct pt_regs *regs, long id),
4
5    TP_ARGS(regs, id),
6
7    TP_STRUCT__entry(
8      __field(  long,    id    )
9      __array(  unsigned long,  args,  6  )
10   ),
11
12   TP_fast_assign(
13     __entry->id  = id;
14     syscall_get_arguments(current, regs, __entry->args);
15   ),
16
17   TP_printk("NR %ld (%lx, %lx, %lx, %lx, %lx, %lx)",
18       __entry->id,
19       __entry->args[0], __entry->args[1], __entry->args[2],
20       __entry->args[3], __entry->args[4], __entry->args[5]),
21
22   syscall_regfunc, syscall_unregfunc
23 );
```

This last macro creates the actual tracepoint that logs all systemcalls, using the metadata registered from `SYSCALL_DEFINEX()` via `SYSCALL_METADATA()` for printing. This also creates an entry in tracefs for each systemcall to enable tracing for that specific systemcall ($TRACEFS/events/syscalls/$SYSCALL).

3.4 Kprobes and Kretprobes

Kprobes are events that can be dynamically placed almost anywhere on the kernel. They use code patching to replace any instruction with a breakpoint instruction; when triggered, it calls one or

more "probe" functions which may perform some tracing operations, and restores the registers when they complete. The instruction that was replaced will be single-stepped (executed or emulated) by the probe, so that the program will continue to run correctly as originally expected. Such instrumentation does not require any special infrastructure on the traced function, which makes kprobes particularly useful and versatile. The kprobe infrastructure is available with the kernel compile option `CONFIG_KPROBES`.

Two different kinds of probes exist: *kprobes* which are often placed at the beginning of functions, and *kretprobes* which are called at the end of the function and may alter the return value. A kprobe or kretprobe sets up a call to two functions, if specified: a *pre-handler*, which is called before single-stepping the replaced instruction, and a *post-handler* which is instead executed after. Figure 3.1 shows these phases in time. Additionally, kprobes can be attached to both regular code and to ftrace's function tracing inftrastructure to be attached to each function.

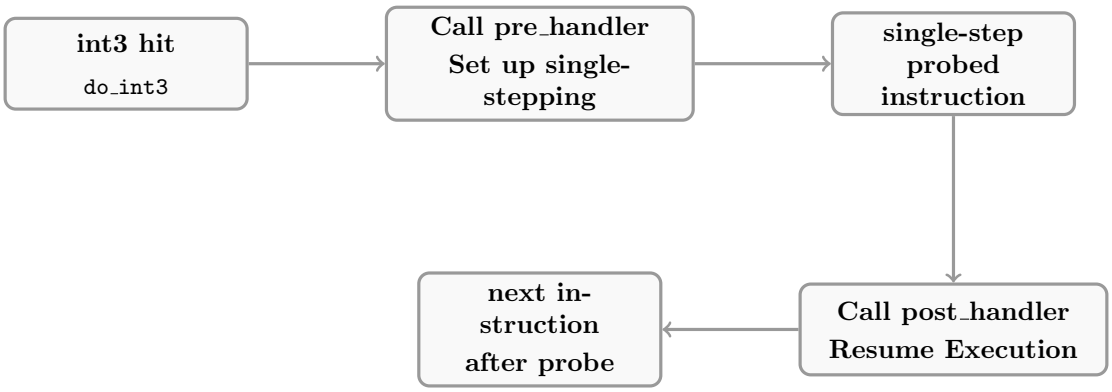


Figure 3.1: Key phases furing the execution a Kprobe.

3.4.1 Kprobe interface

A kprobe is represented by the type `struct kprobe` which is reported below, with comments explaining the various members:

```
typedef int (*kprobe_pre_handler_t) (struct kprobe *, struct pt_regs *);
typedef void (*kprobe_post_handler_t) (struct kprobe *, struct pt_regs *,
               unsigned long flags);

struct kprobe {
  struct hlist_node hlist;
  /* list of kprobes for multi-handler support */
  struct list_head list;
  /*count the number of times this probe was temporarily disarmed */
  unsigned long nmissed;
  /* location of the probe point */
  kprobe_opcode_t *addr;
  /* Allow user to indicate symbol name of the probe point */
  const char *symbol_name;
  /* Offset into the symbol */
  unsigned int offset;
  /* Called before addr is executed. */
  kprobe_pre_handler_t pre_handler;
  /* Called after addr is executed, unless... */
  kprobe_post_handler_t post_handler;
  /* Saved opcode (which has been replaced with breakpoint) */
  kprobe_opcode_t opcode;
  /* copy of the original instruction */
```

```
  struct arch_specific_insn ainsn;
  /*
   * Indicates various status flags.
   * Protected by kprobe_mutex after this kprobe is registered.
   */
  u32 flags;
};
```

A kprobe can be registered and unregistered with the functions `register_kprobe()` and `unregister_kprobe()`. Kprobes can be temporarely enabled with `enable_kprobe()` and disabled with `disable_kprobe()` after registration. The list of registered kprobes is exposed to the user by reading the file `$DEBUGFS/kprobes/list` where `$DEBUGFS` is the location where the *debugfs* filesystem is mounted.

To showcase and explain the kprobe interface, an example kernel module will be analyzed. The module register a kprobe on the `do_sys_openat2()` function which is called by the `openat2` systemcall, and traces the name of the file that the user wants to open:

```
1  #include <linux/module.h>
2  #include <linux/kprobes.h>
3
4  #define MAX_FILENAME_LEN 128
5
6  /* This gets executed before single-stepping the probed instruction */
7  static int hello_kprobe_pre_handler(struct kprobe *p, struct pt_regs *regs)
8  {
9    const char __user *user_filename =
10     (const char __user *)regs_get_kernel_argument(regs, 1);
11   if (!user_filename)
12     return 0;
13
14   char *filename = kmalloc(MAX_FILENAME_LEN, GFP_ATOMIC);
15   if (!filename)
16     return 0;
17
18   if (strncpy_from_user(filename, user_filename, MAX_FILENAME_LEN-1) > 0) {
19     filename[MAX_FILENAME_LEN-1] = '\0';
20     printk(KERN_INFO "Kprobe: openat -> %s\n", filename);
21   }
22
23   kfree(filename);
24   return 0;
25 }
26
27 /* This gets executed after single-stepping the probed instruction */
28 static void hello_kprobe_post_handler(struct kprobe *kprobe,
29                                       struct pt_regs *regs,
30                                       unsigned long flags)
31 { /* does nothing */ }
32
33 struct kprobe kp = (struct kprobe) {
34   .symbol_name  = "do_sys_openat2",
35   .pre_handler  = hello_kprobe_pre_handler,
36   .post_handler = hello_kprobe_post_handler,
37 };
```

```
38
39 static int __init hello_kprobe_init(void)
40 {
41   printk(KERN_INFO "Kprobe: Hello, kprobe!\n");
42
43   int ret = register_kprobe(&kp);
44   if (ret < 0) {
45     printk(KERN_ERR "Kprobe: error registering kprobe\n");
46     return -1;
47   }
48   return 0;
49 }
50
51 static void __exit hello_kprobe_cleanup(void)
52 {
53   unregister_kprobe(&kp);
54   printk(KERN_INFO "Kprobe: unregistered");
55 }
56
57 module_init(hello_kprobe_init);
58 module_exit(hello_kprobe_cleanup);
59
60 MODULE_AUTHOR("Giovanni Santini");
61 MODULE_DESCRIPTION("kprobe example");
62 MODULE_LICENSE("GPL v2");
```

On line 7 and line 28 the kprobe's pre-handler `hello_kprobe_pre_handler()` and post-handler `hello_kprobe_post_handler()` functions are defined. Both pre and post handlers can access the CPU registers via the struct `struct pt_regs`. Looking at the pre-handler, the values of the registers correspond to the argument to the function according to the C ABI for the specific CPU architecture used, since the kprobe was placed in a function. The pre-handler gets the second argument from the function using `regs_get_kernel_argument()` on line 10, which corresponds to the filename, then copies its value from userspace to kernelspace on line 18 and logs it on line 20 using `printk()` which writes it to a special ringbuffer used for kernel prints which is accessible from `$PROCFS/kmsg`. On line 33 the code defines the kprobe `kp` with its pre and post handler, and the location where it should be placed. The specific address of the kprobe will be resolved from the symbol specified in the `symbol_name` member. Finally on line 43 the kprobe is registered when the module is loaded, and unregistered on line 53 when the module is unloaded.

3.4.2 Kprobe implementation

Most of the implementation of the kprobe interface is located in the file `$LINUX/kernel/kprobes.c`. Each registered kprobe is stored in the hash table `kprobe_table` which is indexed by the address where it is placed. Insertion and deletion to this table is guarded by the mutex `kprobe_mutex` to prevent such operations from executing simultaneously. A list of blacklisted address ranges `kprobe_blacklist` is used for areas where kprobes are not allowed to be registered.

An example that showcases the use of `kprobe_table` is the function `get_kprobe()` present in `$LINUX/kernel/kprobes.c:378` which returns the `struct kprobe` from an address where a kprobe was placed:

```
1 struct kprobe *get_kprobe(void *addr)
2 {
3   struct hlist_head *head;
```

```
4   struct kprobe *p;
5
6   head = &kprobe_table[hash_ptr(addr, KPROBE_HASH_BITS)];
7   hlist_for_each_entry_rcu(p, head, hlist,
8           lockdep_is_held(&kprobe_mutex)) {
9     if (p->addr == addr)
10      return p;
11  }
12
13  return NULL;
14 }
```

In the code above, a list of kprobes is retrieved from the hashmap in constant time complexity by using an hash of the pointer where the kprobe is placed. Then, for each kprobe in the list, if its address actually corresponds to the query then it is returned. Otherwise the function returns a `NULL` pointer.

When registering a kprobe regularly (not through ftrace), `register_kprobe()` first retrieves the address of the probed instruction from the symbol, if specified, via `kallsyms_lookup_name()`. This performs a binary search of the symbol name from a compressed array containing a mapping from each symbol to its address. The registration continues by calling `prepare_kprobe()` which sets up internal structures, it then adds the hash of the kprobe to the `kprobe_table` map and calls `arm_kprobe()` only if the kprobe was not set to be disabled by default. This last function is responsible to replace the instrumented instruction in memory with a breakpoint call (`int3` on x86). It finally sets up the emulation of the probed instruction, which emulate it instead of calling it directly, avoiding the need for a second breakpoint to call the post-handler. After registering, if the kernel is compiled with `CONFIG_OPTPROBES`, it tries to optimize the break instruction with a jump. This does not work if the kprobe was based on ftrace. The optimizer logic is quite complex and will not be covered here.

The `int3` breakpoint handler for x86 is defined in `$LINUX/arch/x86/kernel/trace.c`. It calls `do_int3()` which interfaces with the kprobe infrastructure to call the registered kprobe, if present, as shown in the following extract:

```
1  static bool do_int3(struct pt_regs *regs)
2  {
3    int res;
4    /* ... */
5  #ifdef CONFIG_KPROBES
6    if (kprobe_int3_handler(regs))
7      return true;
8  #endif
9    /* ... */
10   return res == NOTIFY_STOP;
11 }
12 NOKPROBE_SYMBOL(do_int3);
```

The `kprobe_int3_handler()` called on line 6 uses two per-cpu variable called `current_kprobe` and `kprobe_ctrlblk` to implement the following logic:

- If `current_kprobe` is empty, meaning that a kprobe is not running, then it sets `curr ent_kprobe`. If specified, it calls the `kp->pre_handler` and proceeds to emulate the probed instruction and call the post handler, or to setup the CPU's instruction pointer to the original instruction. In this case, the instruction pointer will be set when the trap returns, the instruction will be executed, and a new trap will be called with `int3` and

with `current_kprobe` not empty. This is necessary because jump instructions cannot be emulated or executed during the single step since they would.

- if `current_kprobe` is non empty, which happens after single stepping the instruction and entering the trap again from the previous point. This returns to the normal execution by setting the instruction pointer to the original place (just after the probed instruction).

A kretprobe allows the user do register a callback that gets executed when the function returns. To implement this, the kretprobe registers a kprobe `pre_handler_kretprobe` that changes the return address when the function is entered. The return address is set to a trampoline function. On arm, this function triggers a breakpoiont exception, which eventually calls `__kretprobe_trampoline_handler`. This function finally calls the user handler, and sets the exception's return address to the correct one after the probe.

3.5 Perf events

The perf tracing framework has been described previously. This section will examine how it is used to register perf events and how they are accessed from userspace.

Consider the following code of a kernel module registering a perf software event counter through a PMU:

```
1  #include <linux/module.h>
2  #include <linux/kthread.h>
3  #include <linux/perf_event.h>
4  #include <linux/uaccess.h>
5
6  static struct pmu hello_pmu;
7  static atomic64_t global_counter = ATOMIC64_INIT(0);
8
9  static int hello_pmu_init(struct perf_event *event)
10 {
11   if (event->attr.type != hello_pmu.type)
12     return -ENOENT;
13
14   if (is_sampling_event(event))
15     return -EINVAL;
16
17   event->hw.state = PERF_HES_STOPPED | PERF_HES_UPTODATE;
18
19   return 0;
20 }
21
22 static void hello_pmu_read(struct perf_event *event)
23 {
24   local64_set(&event->count, atomic64_read(&global_counter));
25 }
26
27 static void hello_pmu_start(struct perf_event *event, int flags)
28 {
29   event->hw.state &= ~(PERF_HES_STOPPED | PERF_HES_UPTODATE);
30 }
31
32 static int hello_pmu_add(struct perf_event *event, int flags)
33 {
34   u64 now = atomic64_read(&global_counter);
```

```
35
36    local64_set(&event->count, 0);
37    local64_set(&event->hw.prev_count, now);
38
39    if (flags & PERF_EF_START)
40      hello_pmu_start(event, flags);
41
42    return 0;
43  }
44
45  static void hello_pmu_stop(struct perf_event *event, int flags)
46  {
47    hello_pmu_read(event);
48    event->hw.state |= PERF_HES_STOPPED;
49  }
50
51  static void hello_pmu_del(struct perf_event *event, int flags)
52  {
53    hello_pmu_stop(event, flags);
54  }
55
56  static void hello_pmu_enable(struct pmu *pmu) { }
57
58  static void hello_pmu_disable(struct pmu *pmu) { }
59
60  static void hello_perf_increment(void)
61  {
62    atomic64_inc(&global_counter);
63  }
64
65  static int __init hello_perf_init(void)
66  {
67    int ret;
68
69    memset(&hello_pmu, 0, sizeof(hello_pmu));
70
71    hello_pmu.task_ctx_nr  = perf_invalid_context;  /* global context */
72    hello_pmu.capabilities = PERF_PMU_CAP_NO_INTERRUPT;
73    hello_pmu.event_init   = hello_pmu_init;
74    hello_pmu.add          = hello_pmu_add;
75    hello_pmu.del          = hello_pmu_del;
76    hello_pmu.start        = hello_pmu_start;
77    hello_pmu.stop         = hello_pmu_stop;
78    hello_pmu.read         = hello_pmu_read;
79    hello_pmu.pmu_enable   = hello_pmu_enable;
80    hello_pmu.pmu_disable  = hello_pmu_disable;
81
82    ret = perf_pmu_register(&hello_pmu, "hello_pmu", -1);
83    if (ret)
84      return ret;
85
86    pr_info("hello_pmu registered with type %d\n", hello_pmu.type);
87
```

```
88   /* Example increment */
89   hello_perf_increment();
90
91   return 0;
92 }
93
94 static void __exit hello_perf_cleanup(void)
95 {
96   perf_pmu_unregister(&hello_pmu);
97 }
98
99 module_init(hello_perf_init);
100 module_exit(hello_perf_cleanup);
101
102 MODULE_AUTHOR("Giovanni Santini");
103 MODULE_DESCRIPTION("perf event example");
104 MODULE_LICENSE("GPL v2");
```

This module defines a counter `global_counter` on line 7 which is atomically incremented by the function `hello_perf_increment()`. In the module initialization function on line 65, the `hello_pmu` PMU is initialzied with pointers to functions used to manage it and registered on line 82. A notable method is `hello_pmu_read()` on line 22 which is used to read the event. This function simply copies the content of the `global_counter` into the event "count" member. When the module is unloaded, the PMU is unregistered on line 96.

An userspace program that reads the event could be implemented with the following code:

```
1  #include <linux/perf_event.h>
2  #include <sys/syscall.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <stdint.h>
6  #include <stdio.h>
7  #include <stdlib.h>
8
9  int main(int argc, char **argv)
10 {
11   struct perf_event_attr attr;
12   int fd;
13   uint64_t count;
14
15   memset(&attr, 0, sizeof(attr));
16   attr.type     = 11; /* depends on the specific event */
17   attr.config   = 0;
18   attr.disabled = 0;
19   attr.inherit  = 0;
20
21   /*
22   int syscall(SYS_perf_event_open, struct perf_event_attr *attr,
23               pid_t pid, int cpu, int group_fd, unsigned long flags);
24   */
25   fd = syscall(__NR_perf_event_open, &attr, -1, 0, -1, 0);
26   if (fd < 0) {
27     perror("perf_event_open");
28     return EXIT_FAILURE;
```

```
29    }
30
31    if (read(fd, &count, sizeof(count)) != sizeof(count)) {
32      perror("read");
33      return 1;
34    }
35
36    printf("Current counter value: %lu\n", count);
37
38    close(fd);
39    return EXIT_SUCCESS;
40  }
```

This program initializes a `struct perf_event_attr` on line 16 to 19 with the information identifying the particular perf event registered in the module, then calls the `perf_event_open()` systemcall on line 25 and reads the counter from the returned file descriptor using the systemcall `read()` on line 31.

4 eBPF

Dynamic event tracing using kprobes gave a powerful tool to developers since they could hook custom logic anywhere in the kernel. However, placing a kprobe requires loading a kernel module which could meaningfully break the system in case of bugs: since kernel modules run as kernel code, they have the same capabilities as the kernel, posing a great security risk. The same issues are presented when extending the tracing infrastructure or other parts of the kernel in order to fit specific needs and run custom logic. The need for an alternative solution to extend the kernel with a more sound security model resulted into the modern eBPF subsystem, which is the subject of this chapter.

4.1 Introduction and overview

The *eBPF subsystem* provides the infrastructure to load custom programs, called *eBPF programs*, by a *loader* program into the kernel, but only after being checked by a *verifier* which enforces a set of security requirements onto the program, making sure that the eBPF program interacts with the rest of the operating system only through a set of well-defined APIs provided by the eBPF subsystem. After being loaded, eBPF programs are executed whenever a certain event happen, which is determined by the *type* of the program; for example, an eBPF program may be attached to a kprobe or a tracepoint, and be invoked when that kprobe is fired. Many other hook possibilities are available, such as when a packet is received on the network, or when the scheduler needs to take a scheduling decision; the eBPF program can then implement custom logic to handle the event, but only though a well-defined interface. The type of the program also determines which APIs are available to the program. eBPF programs are written in a dedicated instruction set architecture and just-in-time compiled into native instructions. User-to-eBPF and eBPF-to-eBPF interactions are handled via shared memory regions called *maps* which provide different types of data structures for the programs. These components are described in more detail in the rest of the chapter.

eBPF evolved from the *Berkeley Packet Filter* (BPF), which allowed to load small programs to filter network requests. Modern *extended* BPF can do much more than packet filtering, so the acronym lost its meaning. The legacy BPF system is now referred to as *classic* BPF or *cBPF*.

An user-space program can load eBPF programs and create maps via the `bpf()` systemcall. The function signature is the following:

```
int bpf(int cmd, union bpf_attr *uattr, unsigned int size,
        struct bpf_common_attr *uattr_common, unsigned int size_common);
```

Where the "cmd" argument specifies the operation what the user wants to perofrm, and the contents of `unionbpf_attr`.

To develop eBPF programs, the officially maintained *libbpf* library is often used, providing a set of useful functionalities to generate loaders and bindings for the program.

4.2 Instruction Set

eBPF programs are written in the eBPF *instruction set architecture* (ISA), often referred to simply as BPF ISA. The ISA defines the set of instructions that an eBPF program uses and that are expected to be supported by an eBPF runtime. A compiler may be used to generate the BPF instructions from an higher-level language such as C or Rust to make the development of eBPF program easier for developers.

For example, the figure below shows C code on the left and an human readable form of BPF bytecode on the right compiled with clang 23.1.0:

```
int sum_foo(int a, int b) {
    return foo(a + b);
}
```

```
sum_foo(int, int):
        *(u32 *)(r10 - 4) = w1
        *(u32 *)(r10 - 8) = w2
        w1 = *(u32 *)(r10 - 4)
        w2 = *(u32 *)(r10 - 8)
        w1 += w2
        call foo(int)
        exit
```

Each line on the right corresponds to a BPF assembly instruction. The term `w1`, `w2` represent the first word of the register number 1 and 2 respectively, and `r10` is the full register number 10.

4.2.1 Standardization

Standardizing the BPF ISA is significantly important to allow compiler developers, tools developers, eBPF runtime developers and ASIC designers to agree to a common reference. The formal IETF standardization effort was started from earlier drafts in October 2023 with RFC 9669 [11]; the RFC was finally standardized one year later and updated multiple times since the first version. All the information presented in this section are referring to the RFC as last updated in 20 May 2026.

Controversially, the RFC does not standardize the platform-specific application binary interface (ABI) because a common consensus was not reached, hence it was preferred to be left out of the standard. This includes various details such as: which registers are saved across a call, which registers contain the frame pointer, how large the stack is, and other specifics. Further standardization is left for "future work"; however, the Linux BPF ABI is well explained in the kernel documentation and any BPF runtime that wishes to be compatible with the Linux implementation must follow it.

4.2.2 Instructions and encoding

The standard specifies two instruction encodings: one using 64 bits and another one using 128 bits. Instructions are assigned to a named conformance group, which is a set of instructions or other requirements identified with a short ASCII label, such as "base32". An implementation of the BPF ISA must support the "base32" conformance group, but may or may not support other groups.

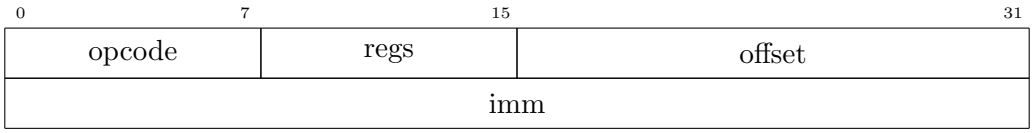


Figure 4.1: 64 bit instruction encoding.

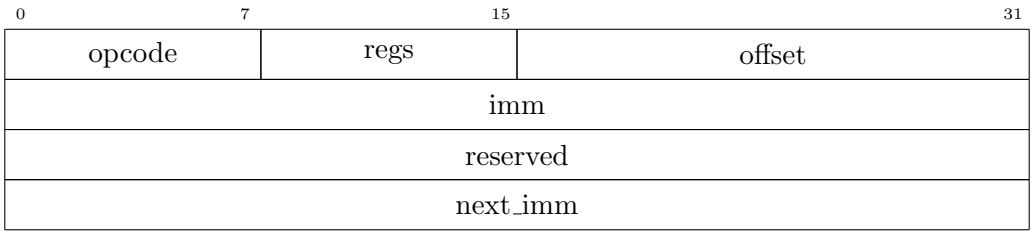


Figure 4.2: Extended 128-bit instruction encoding.

Figure 4.1 shows the encoding of 64 bit instructions. The various field are the following:

- `opcode`: the operation to perform, encoded as a *class* part and a *specific* part which identifies the instruction on the class.

- `regs`: source and destination register numbers. Some instructions might reuse this fields for other purposes.
- `offset`: signed integer offset used with pointer arithmetic, or for other purposes.
- `imm`: signed integer immediate value.

Figure 4.2 shows the encoding for extended 128 bit instructions. Compared to 64 bit instructions, it adds a `reserved` 32 bit field which is set to 0, and a `next_imm` 32 bit field for a second signed integer immediate value.

The instruction set is a reduced one in the number of instruction, resembling RISC-V. It has various instruction classes such as arithmetic operations `ADD` and `MUL`, jump instructions `JMP` and `JEQ`, load and store from memory, etc. Additionally, the BTF ISA supports atomic operations for load and store instructions, which ensure that the operation cannot be interrupted or corrupted by other accesses to the same memory region by another BPF program or other actors. Other than atomic `ADD`, `OR`, `AND` and `XOR`, BPF provides `XCHG` for atomic exchange, and `CMPXCHG` for atomic compare and exchange. A complete list of the encoding of each instruction is found in the RFC.

4.2.3 Helper Functions

The RFC introduces the concept of *helper functions* which provide functionalities implemented by the underlying platform. Notably, helper functions are not standardized by RFC9669, but they are documented in the Linux kernel, which will be discussed in a later chapter. To call an helper function, the jump instruction must be executed with its source register as $0x0$ or $0x2$, with a platform-specific `ID` identifying a specific function.

4.2.4 Maps and platform variables

RFC9669 defines "maps" as shared memory regions accessible by BPF programs on some platforms. The particular semantics of a map are not defined by the standard. Linux supports many kinds of maps which implement various data structures such as hash maps, arrays, ring buffers etc which provide shared storage across userspace and kernelspace, and among multiple executions of the same eBPF program or multiple eBPF programs. Linux maps will be discussed later.

The standard also defines "platform variables" which are memory regions identified by integer ids exposed by the runtime. They are different than maps as they are used directly by the program instead of going through helper functions, conceptually platform variables can be thought of as global variables.

4.2.5 Compiler Support

Clang 11+ and GCC 12.1+ support compiling C to BPF bytecode. Compilers for other languages also support the BPF ISA, such as the rust compiler. Atomic operations which map to atomic instructions can be specified with intrinsics.

4.3 The verifier

After loading and eBPF program, the BPF code is *just in time* (JIT) compiled to native machine instructions to be executed as kernel code in a later moment. This would give eBPF programs the ability to run almost arbitrary code in kernelspace, with important security implications on confidentiality, integrity and availability. Without a proper security model, an eBPF program could potentially read or write arbitrary kernel memory (confidentiality and integrity issue) or invoke other kernel functions and alter the execution of the operating system to halt it (availability issue). For these reasons, the eBPF security model determines strict guardrails and limitations to what a program is allowed to do. The model discriminates a program as *safe* if it meets all security requirements, or as *unsafe* otherwise; the user can decide to disallow the loading of any unsafe programs, which is the default policy on most Linux distributions. This

model is fundamental to the success and wide adoption of eBPF in production and security-critical environments. Other security considerations and policies will be discussed in a later section, such as user permissions and chain of trust.

The following is a non-exhaustive list of security requirements for safe eBPF programs:

- Programs must always terminate: no infinite loops or infinite recursion is allowed (availability issue).
- Programs cannot to read or write arbitrary memory (confidentiality, integrity issue).
- Network programs are not allowed to access memory outside of packet bounds (confidentiality, integrity issue).
- Programs are not allowed to deadlock (availability issue).
- Programs are not allowed to read uninitialized memory (confidentiality issue).

Notably, the safety requirements are not standardized and may change across kernel versions. The reason is that the eBPF ecosystem is rapidly evolving, hence putting effort in standardization is not practical or wanted at the current state.

The requirements are checked at program load time instead of at runtime, to trade runtime performance penalties with load-time overhead. If a requirement cannot be fully checked at load-time, then the eBPF program is rejected or a method to perform the checks dynamically is provided, although only for a limited set of functionalities (dynamic pointers). The program that checks the requirements is known as the eBPF *verifier*: given a BPF program, the output of the verifier will determine if the program is considered safe or not. Therefore, the verifier is a critical part of the eBPF architecture and its implementation has been widely scrutinized. Given the complexity of checking some of the requirements at load time, the verifier implementation is highly complex. Just the file `$LINUX/kernel/bpf/verifier.c`, which implements most of the logic, is about 20000 lines of code long, although solutions to simplify and modularize it are being discussed by its developers.

The verifier implementation can be divided in two separate moments. First, it builds a control flow graph of the program, looks for unreachable instructions and identifies back-edges (loops) among other correctness checks. Second, it descends every possible execution path in the graph from the first instruction and looks for unsafe operations, such as out of bounds array accesses and others. It does so by simulating the execution and keeping track of the state of each register and the range of possible values it can hold (*value tracking*). If, during the simulation, a state that was already verified is encountered during a backward jump, then the verifier considers this path as safe and does not re-simulate the code again (*pruning*) which greatly speeds up verification. If any of the safety requirements are not met in any particular state, then the program is rejected. A more in depth look is now provided.

4.3.1 Initial checks and preparations

A program's verification starts in `bpf_check()` after the `bpf()` systemcall is executed with the `BPF_PROG_LOAD` command. Upon verification entry, the verifier allocates an environment structure of type `struct bpf_verifier_env`. The environment holds all the state of the verifier for a particular program, such as jump history, register states (`struct bpf_reg_state`) and the history of previously explored states, among other information. Additionally, it contains a log buffer of type `struct bpf_verifier_log` which logs the state of the program after each instruction; this is particularly useful to debug verification issues. Since the verification environment is allocated per-verification, multiple verifications on different programs can happen simultaneously. Parallel verifications are allowed only if the program's token (see later) has the `CAP_BPF` capability, since unprivileged users could exhaust kernel memory and degrade performance by requesting many verifications. A token is a delegation mechanism where a privilege process can transfer capabilities to an unprivileged one via a token, which is then passed to the eBPF program. This will be discussed more in a following chapter.

After some initializations, the verifier proceeds to perform several checks and compute various values and structures in the context. Some of these operations are reported below:

- `check_subprogs()`: check that all jumps are within the same subprogram and all subprograms end (a program can contain multiple subprograms).
- `check_and_resolve_insns()`: check that instructions are valid, replace map IDs with actual map pointers
- `bpf_check_cfg()`: a non-recursive depth-first-search to detect loops in the BPF program. Each instruction, which is represented as a *vertex* in a directed graph, can have either a *discovered*, *explored*, *fallthrough* or *branch* label. The difference between discovered and explored is that a discovered instruction did not check all the edges yet, while an explored one was fully checked. Each *edge* can either be discovered or not. While traversing the graph, the verifier assigns these labels to the instructions. If the DAG explores any instruction already marked as discovered, a back edge is detected and annotated. This information is used during emulation to avoid emulating the entire loop with a technique knows as *pruning* which will be explained later.
- `bpf_prune_dead_branches()`: rewrite conditional branches that have constant outcomes into unconditional jumps, so that dead code is not checked.
- `sort_subprogs_topo()`: sort the subprograms topologically
- `bpf_compute_scc()`: : Use a non-recursive adaptation of Tarjan's algorithm for strongly connected components computation.

4.3.2 Register types

To proceed with the next phase of verification, the verifier uses a system of *register types*: during simulation, each register may have a type assigned to it. Types provide key information about the type a register holds to help the verifier make certain decisions. Performing operations on registers may modify their type, and some operations are not allowed wht certain types. For example, load and store instructions are allowed only with registers of type `PTR_TO_CTX`, `PTR_TO_MAP` and `PTR_TO_STACK`.

The set of available types is defined in `enum bpf_reg_type` present in `$LINUX/include/linux/bpf.h`. The following is an extract of the enum for context:

```
enum bpf_reg_type {
  NOT_INIT = 0,     /* nothing was written into register */
  SCALAR_VALUE,     /* reg doesn't contain a valid pointer */
  PTR_TO_CTX,     /* reg points to bpf_context */
  CONST_PTR_TO_MAP,   /* reg points to struct bpf_map */
  PTR_TO_MAP_VALUE,   /* reg points to map element value */
  PTR_TO_MAP_KEY,     /* reg points to a map element key */
  PTR_TO_STACK,     /* reg == frame_pointer + offset */
  PTR_TO_PACKET_META,   /* skb->data - meta_len */
  PTR_TO_PACKET,     /* reg points to skb->data */
  PTR_TO_PACKET_END,   /* skb->data + headlen */
  PTR_TO_FLOW_KEYS,   /* reg points to bpf_flow_keys */
  PTR_TO_SOCKET,     /* reg points to struct bpf_sock */
  PTR_TO_SOCK_COMMON,   /* reg points to sock_common */
  /* ... */
}
```

4.3.3 Program simulation

The verification proceeds to the next major phase: program simulation. The simulation descends all possible paths of the program to check for correctness.

The simulation begins with `do_check_main()`. At the very start, the only valid register to be accessed is `R1` of type `PTR_TO_CTX` which can be used to read a `struct bpf_context` containing various information about the program depending on the program type. The verifier keeps the state of the simulation by allocating a `struct bpf_verifier_state` and `structbpf_func_state`. It proceeds to call `do_check` which calls `do_check_insn` for each instruction of the program; this performs all checks for an instruction, such as bound checks and type compatibility checks. While iterating over all the instructions, the verifier keeps a stack of states: whenever it encounters a conditional jump, it pushes a new state in the stack, and when an exit condition is reached, the top state is popped and simulated, until all states are checked (the stack is empty). If all checks were successful, the verifier calls `do_check_subprogs()` which lazily verifies all global functions. This is necessary because, during verification, all functions are assumed to be already verified successfully and checked for signature match only.

4.3.4 Range analsyis

A powerful feature of the verifier is an abstract interpretation technique known as *range analysis* [3]. For each registers, the verifier keeps track of the range of its possible values (minimum and maximum). This is important to verify bounded accesses to arrays and other checks. For example, consider the following C code:

```
1 unsigned int buff[100];
2 unsigned int i, x, y; /* initialized somewhere randomly */
3
4 if (i < 10) {
5   x = buff[i];   /* OK, i is within [0, 10) */
6 } else {
7   y = buff[i];   /* Rejected, i is out of bounds [0, UINT_MAX) */
8 }
```

On line 1, a buffer with 10 elements is created. Every time the buffer is accessed, the verifier will check that the index used, i in this case, is always in bounds my tracking the range of values in the register that stores its value. If the simulation takes the code path where the if statement on line 4 is evaluated to true, then value that i holds must have an upper limit of 10 because of the condition in the if statement. If the verifier cannot ensure that i does not hold a value smaller than the size of the buffer, then the program is rejected as shown on line 7. The implementation of range analysis has been formally verified by [12]

Additionally, the verifier knows which offsets inside structs can be accessed. For example, when evaluating the instruction $bpf_ld\ R0 = *(u32*)(R6+8)$, a check will be performed to ensure that the address $R6+8$ can be loaded, and if $R6$ is of type `PTR_TO_STACK` then an additional check will be made to ensure that this address is within stack bounds $[-MAX_BPF_STACK, 0)$. It would also be checked that such stack location was written to previously so that it would be uninitialized.

4.3.5 State pruning

Some states are saved for the entire verification, for example when performing a jump to a previous instruction identified by a back-edge during the initial phase of the verifier. This is used for an optimization called *pruning*: if the current state of the verification was already visited and verified, then the verifier prunes the current state, meaning that it considers this execution path as verified and proceeds to pop the stack to simulate other branches. This greatly speeds up verification time since some "while" or "for" loops can be fully pruned instead of emulating every cycle.

After successful verification, the kernel finally proceeds to JIT compile the program into native assembly. The call stack looks like the following:

```
bpf_check
  __bpf_prog_select_runtime
    bpf_prog_jit_compile
      bpf_int_jit_compile (arch/x86/net/bpf_jit_comp.c)
        do_jit
```

4.4 Program Types

Each eBPF program is of a certain type, which determines what resources and functions the program has access to, and when the program is executed. The possible types are enumerated in `enum bpf_prog_type` in `$LINUX/include/uapi/linux/bpf.h`, an extract is reported below:

```
enum bpf_prog_type {
  BPF_PROG_TYPE_UNSPEC,
  BPF_PROG_TYPE_SOCKET_FILTER,
  BPF_PROG_TYPE_KPROBE,
  BPF_PROG_TYPE_SCHED_CLS,
  BPF_PROG_TYPE_SCHED_ACT,
  BPF_PROG_TYPE_TRACEPOINT,
  BPF_PROG_TYPE_XDP,
  BPF_PROG_TYPE_PERF_EVENT,
  BPF_PROG_TYPE_CGROUP_SKB,
  /* ... */
};
```

By convention, loaders based on *libbpf* specify the type of the program by putting the program's binary code in a specific ELF section of the binary, which depends upon the program type. For example, a program of type `BPF_PROG_TYPE_CGROUP_DEVICE` must be put in the `cgroup/dev` ELF section, and a program of type `BPF_PROG_TYPE_CGROUP_SKB` is put in the `cgroup/skb` ELF section. This trick allows to specify multiple programs of different types in the same ELF binary, and it is used by libbpf to provide a set of convenient APIs for eBPF program developers.

4.4.1 Attach points

After loading, verifying and JIT compiling the program, it needs to be attached to some event or other object; the attach point depends on the type of the program. For example, the program will be attached to a perf event if the program type is `BPF_PROG_TYPE_PERF_EVENT` or to a function through a kprobe if the program type is `BPF_PROG_TYPE_KPROBE`. The list of possible attachment is enumerated in `enum enum bpf_attach_type`. When attaching the program, an object known as a *link* is created which identifies the relationship between a program and its attachment point, and manages its lifetime. The execution of the program depends on its attachment point, for example a program that is attached to a tracepoint will be executed every time that tracepoint is fired. The program will also be able to access the information of the tracepoint if it is attached to it, so that it can implement specific custom logic to react to the event.

4.4.2 Trampolines

To create certain types of attachments, special trampolines are used, which rely on the dynamic ftrace infrastructure as explained in a previous chapter, to allow kernel code to call into BPF programs with practically zero overhead using code patching. Such special attachments are `BPF_TRACE_FENTRY`, `BPF_TRACE_FEXIT`, `BPF_MODIFY_RETURN`, `BPF_TRACE_FSESSION`, `BPF_TRACE_FENTRY_MULTI`, `BPF_TRACE_FEXIT_MULTI`, `BPF_TRACE_FSESSIONS_MULTI`. When an

eBPF program that uses a trampoline is linked, the function `bpf_trampoline_update()` is called which creates a page via `arch_prepare_bpf_trampoline()`, setting up the stack environment for the eBPF program including its arguments, its execution, restoring registers and finally returning to the target function.

Some example eBPF program types are now analyzed.

4.4.3 BPF_PROG_TYPE_XDP

Express Data Path (XDP) programs are called by a network device driver for every incoming packet received, before allocating the `struct sk_buff` and before passing the packet up the network stack. An XDP program can access a `struct xdp_md` representing a single packet, and has the capability to manipulate, drop, redirect or accept it. The early invocation of the eBPF program in the lifetime of a packet, before any other allocations or processing, makes XDP programs particularly useful for a range of use cases: DDoS protection, load balancing, high-throughput packet filtering, etc.

The `struct xdp_md` is reported below:

```
1  struct xdp_md {
2    __u32 data;
3    __u32 data_end;
4    __u32 data_meta;
5    // Below access go through struct xdp_rxq_info
6    __u32 ingress_ifindex; // rxq->dev->ifindex
7    __u32 rx_queue_index;  // rxq->queue_index
8
9    __u32 egress_ifindex;  // txq->dev->ifindex
10 };
```

Where `data` and `data_end` on lines 2 and 3 are pointers to memory where the packet can be accessed. The verifier will ensure that only memory in the range $[data; data_end)$ will be dereferenced.

If a network interface driver supports XDP, it will call it when processing packets via NAPI, the call stack looks like the following:

```
__napi_busy_loop
  -> napi_poll         # Pool new packets
    -> __napi_poll
      -> struct napi_struct->poll
        # Driver specific functions for the Intel i40e
        -> i40e_napi_poll
          -> i40e_clean_rx_irq
             -> i40e_run_xdp
               -> bpf_prog_run_xdp  # Calls XDP program
```

For example, an XDP eBPF program that blocks all TCP/IP traffic reaching port 1234 looks like this:

```
1  #include <vmlinux.h>
2  #include <bpf/bpf_core_read.h>
3  #include <bpf/bpf_endian.h>
4  #include <bpf/bpf_helpers.h>
5
6  char LICENSE[] SEC("license") = "GPL";
7
8  /* IEEE 802.3 Ethernet magic constants, found in
```

```
 * linux/include/uapi/linux/if_ether.h */
#define ETH_P_IP   0x0800    /* Internet Protocol packet */

/* IANA protocol numbers */
#define TCP_PROTO_NUMBER  0x06

SEC("xdp")
int handle_xdp(struct xdp_md *ctx)
{
  void *data_end = (void*)(long)ctx->data_end;
  void *data     = (void*)(long)ctx->data;

  /* ethernet frame */
  struct ethhdr *eth = data;
  if ((void*)(eth+1) > data_end)
    return XDP_PASS;

  if (eth->h_proto == bpf_htons(ETH_P_IP)) {

    /* ip packet */
    struct iphdr *ip = (void*)(eth + 1);
    if ((void*)(ip + 1) > data_end)
      return XDP_PASS;

    if (ip->protocol != TCP_PROTO_NUMBER) /* Only interested in TCP */
      return XDP_PASS;

    /* TCP segment */
    struct tcphdr *tcp = (void*)ip + ip->ihl * 4;
    if ((void*)(tcp + 1) > data_end)
      return XDP_PASS;

    /* Drop the packet */
    if (tcp->dest == bpf_htons(1234))
      return XDP_DROP;
  }

  /* Accept the packet */
  return XDP_PASS;
}
```

4.4.4 BPF_PROG_TYPE_TRACEPOINT

Tracepoint eBPF programs use the pre-existing tracepoint infrastructure to call eBPF programs. A program can be attached to a tracepoint and access the context that the regular tracepoint captures.

For example, the following program traces all calls to the `write()` systemcall from a specific process specified by the loader via the variable `target_pid`:

```
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>

char LICENSE[] SEC("license") = "GPL";

```

```
int target_pid = 0;

SEC("tp/syscalls/sys_enter_write")
int handle_tp(struct trace_event_raw_sys_enter *ctx)
{
    int pid = bpf_get_current_pid_tgid() >> 32;

    if (pid != target_pid)
        return 0;

    bpf_printk("BPF: write() syscall called from PID %d.\n", pid);

    return 0;
}
```

4.4.5 BPF_PROG_TYPE_LSM

LSM eBPF programs are attached to LSM hooks. Whenever the hook is fired, the eBPF program is executed and can access whatever the LSM hook provides. LSM eBPF programs can be chained together to support the enforcement of multiple policies, and each program can access the security decision from the previous one and decide to propagate it to the next one or to ignore it. For example, the following eBPF program logs and blocks all calls to the `bpf()` systemcall:

```
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

char LICENSE[] SEC("license") = "GPL";

#define EPERM 1

/* The "ret" argument is the return value from the previous BPF program or
 * 0 if it is the first hook */
SEC("lsm/bpf")
int BPF_PROG(lsm_bpf, int cmd, union bpf_attr *attr,
             unsigned int size, int ret)
{
    if (ret != 0)
        return ret;

    bpf_printk("BPF LSM: blocked bpf()");
    return -EPERM;
}
```

4.5 Helper functions and Kfuncs

To perform special operations and to interact with the rest of the operating system, eBPF are allowed to use special tailored functions for this purpose, called *helper functions*. Not all programs can use all helper functions: the eBPF program type determines the set of helpers that can be called; this ensures that an eBPF program is only allowed to interact with a limited set of the kernel's subsystems. Helper functions are considered User Space API (UAPI), meaning that the API is stable and will be kept forward compatible on future kernel versions. A full list of helpers is documented in the *bpf-helpers(7)* man page.

Additionally, eBPF programs are allowed to call another set of special functions: *kfuncs*. These functions can be used to access other infrastructure in the kernel, but they differ from helper functions as they are not considered UAPI. This means that an eBPF program that uses kfuncs may not be compatible with new versions of the kernel, therefore defensive programming techniques are advised, such as by supporting alternatives in the eBPF program if a function is not available. Kfuncs are now the preferred method for adding new functionalities to eBPF programs since they are not expected to be supported indefinitely: this makes it possible to refactor and improve them with less effort and reduce code debt. An example of kfunc registration is now analyzed.

The following code defines the kfunc `bpf_list_add()`:

```
1  __bpf_kfunc_start_defs();
2
3  __bpf_kfunc int bpf_list_add(struct bpf_list_head *head,
4             struct bpf_list_node *new,
5             struct bpf_list_node *prev__nonown_allowed,
6             struct btf_struct_meta *meta, u64 off) {
7    /* ... */
8  }
9
10 __bpf_kfunc_end_defs();
```

The function needs to be declared between `__bpf_kfunc_start_defs()` and `__bpf_kfunc_end_defs()` on line 1 and 9 respectively, which remove some compiler warnings. The `__bpf_kfunc` specifies some linking options. They expand with the following code:

```
1  #define __bpf_kfunc __used __retain __noclone noinline
2
3  #define __bpf_kfunc_start_defs()                 \
4    __diag_push();                       \
5    __diag_ignore_all("-Wmissing-declarations",              \
6          "Global kfuncs as their definitions will be in BTF");\
7    __diag_ignore_all("-Wmissing-prototypes",              \
8          "Global kfuncs as their definitions will be in BTF")
9
10 #define __bpf_kfunc_end_defs() __diag_pop()
11 #define __bpf_hook_start() __bpf_kfunc_start_defs()
12 #define __bpf_hook_end() __bpf_kfunc_end_defs()
```

After declaring the function, it needs to be registered so that the verifier knows about it, which ensures that only valid kfuncs are called. This is done with the macro `BTF_ID()` which is used in the following way:

```
1  BTF_ID(func, bpf_list_add)
```

The macro expands to `____BTF_ID`:

```
1  #define BTF_IDS_SECTION ".BTF_ids"
2
3  #define ____BTF_ID(symbol, word)      \
4  asm(              \
5  ".pushsection " BTF_IDS_SECTION ",\"a\";       \n"  \
6  ".local " #symbol " ;                          \n"  \
```

```
7  ".type  " #symbol ", STT_OBJECT;                \n"  \
8  ".size  " #symbol ", 4;                         \n"  \
9  #symbol ":                                      \n"  \
10 ".zero 4                                        \n"  \
11 word              \
12 ".popsection;                                   \n");
```

For the kfunc, an unique label in the `.BTF_ids` section is created (line 5), which is initially filled with 4 bytes of zeros (line 10) during compilation and resolved later during the linking phase by a special tool named `resolve_btfids` which is invoked by the kernel build infrastructure. These 4 byes is filled with the BTF ID of the kfunc, used by the verifier to match it with the eBPF program. Moreover, additional information about the arguments of the kfunc can be specified for the verifier via a technique known as *annotations*, such as whether a null pointer is accepted or that only a known constant value can be passed.

4.6 Maps

RFC 9669 for the BPF Instruction Set Architecture defines the concept of maps as "shared memory regions accessible by BPF programs on some platforms". They were included in the standard because maps are heavily used by the Linux eBPF implementation as one of the main mechanisms to interact between the eBPF program and userspace, and among eBPF programs. However, the RFC does not standardize any specific map interface, leaving this important detail to the implementation. In this section we examine how maps are implemented in the Linux kernel. Due to the number of types of maps (more than 30), the specifics of every maps are not described, their interface is documented in the `bpf` man page.

Many types of maps exist, specifying different semantics and memory layouts. These are enumerated in the type `enum bpf_map_type`, and are mostly used as generic data structures for storage of different types of data. Maps generally provide an API to implement stacks, queues, ring buffers, arenas etc. As maps are reference counted, they will stay in memory as long as there are users of them.

Any map implements a set of operations specified in the `struct bpf_map_ops` type as shown below from `$LINUX/include/linux/bpf.h`:

```
1  struct bpf_map_ops {
2    /* funcs callable from userspace (via syscall) */
3    int (*map_alloc_check)(union bpf_attr *attr);
4    struct bpf_map *(*map_alloc)(union bpf_attr *attr);
5    void (*map_release)(struct bpf_map *map, struct file *map_file);
6    void (*map_free)(struct bpf_map *map);
7    int (*map_get_next_key)(struct bpf_map *map, void *key, void *next_key);
8    void (*map_release_uref)(struct bpf_map *map);
9    /* ... */
10   /* funcs callable from userspace and from eBPF programs */
11   void *(*map_lookup_elem)(struct bpf_map *map, void *key);
12   long (*map_update_elem)(struct bpf_map *map, void *key, void *value, ...);
13   long (*map_delete_elem)(struct bpf_map *map, void *key);
14   long (*map_push_elem)(struct bpf_map *map, void *value, u64 flags);
15   long (*map_pop_elem)(struct bpf_map *map, void *value);
16   long (*map_peek_elem)(struct bpf_map *map, void *value);
17   void *(*map_lookup_percpu_elem)(struct bpf_map *map, void *key, u32 cpu);
18   int (*map_get_hash)(struct bpf_map *map);
19   /* ... */
20 }
```

The various functions such as `map_alloc()` on line 4 and `map_free()` on line 6 and many others are called at various moments during the map's lifetime. The functions `map_lookup_elem()`, `map_update_elem()` and similar ones are used to modify its contents. The specific implementation of these functions depends on how the specific map wants to handle the data.

After defining the various operations for a map and defining static storage for a `structbpf_maps_ops`, the map is registered with all maps by appending its name to the `$LINUX/include/linux/bpf_types.h` in the macro `BPF_MAP_TYPE()`. All map entries from this file will be saved in a global array of maps in the following manner:

```
1  static const struct bpf_map_ops * const bpf_map_types[] = {
2  #define BPF_PROG_TYPE(_id, _name, prog_ctx_type, kern_ctx_type)
3  #define BPF_MAP_TYPE(_id, _ops) \
4    [_id] = &_ops,
5  #define BPF_LINK_TYPE(_id, _name)
6  #include <linux/bpf_types.h>
7  #undef BPF_PROG_TYPE
8  #undef BPF_MAP_TYPE
9  #undef BPF_LINK_TYPE
10 };
```

The header `linux/bpf_types.h` in line 6 will be included inside the `bpf_map_types` array, and each `BPF_MAP_TYPE()` macro will be expanded to place each map's operations in the array based on it's ID on line 4.

To create a map from userspace, the `bpf()` systemcall is used:

```
1  int bpf_create_map(enum bpf_map_type map_type,
2                 unsigned int key_size,
3                 unsigned int value_size,
4                 unsigned int max_entries)
5  {
6      union bpf_attr attr = {
7          .map_type    = map_type,
8          .key_size    = key_size,
9          .value_size  = value_size,
10         .max_entries = max_entries
11     };
12
13     return bpf(BPF_MAP_CREATE, &attr, sizeof(attr));
14 }
```

Lookups and other operations are also executed through the `bpf()` systemcall:

```
1  int bpf_lookup_elem(int fd, const void *key, void *value)
2  {
3      union bpf_attr attr = {
4          .map_fd = fd,
5          .key    = ptr_to_u64(key),
6          .value  = ptr_to_u64(value),
7      };
8
9      return bpf(BPF_MAP_LOOKUP_ELEM, &attr, sizeof(attr));
10 }
```

4.7 BTF and CO-RE

In order to help developers to debug a program, compilers support the generation of debugging information from the source code. Such information may include: names, types, function signatures and any language-specific meta data that can be used to pretty-print or to reference the original source code. On Linux, a standard format used by userspace programs for encoding debug information is *DWARF*. For example, both gcc and clang compilers generate debug information in the DWARF format if the `-g` flag is specified. Many other compilers use DWARF as the default encoding for debug information, such as the *go* and *rust* compiler; this is because DWARF was designed to be a generic and expressive format that can be applied with many languages. Developers of eBPF programs are also interested in expressive debug information, however, the DWARF format was considered too complex and verbose due to its design decisions; inspired by the minimalistic and compact design of *Compact C Type Format* (CTF) developed by Sun Microsystems in Solaris 9, a custom format was developed specifically for eBPF: the *BPF Type Format* (BTF). BTF encodes information for all the necessary types, such as unions, structs, enums, typesefs, etc, and can be used to pretty-print eBPF maps, functions and structs, among other things. Compilers usually embeds them in the `.BTF` ELF section of the eBPF program. The details of the format are documented in the Linux kernel documentation.

For example, consider the following code:

```
1 struct foo {
2   int a;
3   int b;
4   unsigned c:15;
5 } __attribute__((preserve_access_index));
6
7 enum bar { U, V };
```

Given the code above, clang will generate the following BTF information, qpshown in an human readable form:

```
1  [2] STRUCT 'foo' size=8 vlen=2
2         'a' type_id=3 bits_offset=0
3         'b' type_id=3 bits_offset=32
4         'c' type_id=4 bits_offset=64 bitfield_size=15
5  [3] INT 'int' size=4 bits_offset=0 nr_bits=32 encoding=SIGNED
6  [4] INT 'unsigned int' size=4 bits_offset=0 nr_bits=32 encoding=(none)
7  ...
8  [16] ENUM 'bar' encoding=UNSIGNED size=4 vlen=2
9         'U' val=0
10        'V' val=1
```

Form the clang output we can see that the name and the size of the struct is saved in line 1, along with the information of its members and their offsets. Each type is assigned an unique number `type_id` which is used to reference the type.

Indeed each type's debug information is encoded in the following compact format, from `$LINUX/include/uapi/linux/btf.h`:

```
1 struct btf_type {
2         __u32 name_off;
3         /* "info" bits arrangement
4          * bits  0-23: vlen (e.g. # of struct's members)
5          * bits 24-30: kind (e.g. int, ptr, array...etc)
6          * bit     31: kind_flag, currently used by
```

```
7           *              struct, union, enum, fwd, enum64,
8           *              decl_tag and type_tag
9           */
10          __u32 info;
11          /* "size" is used by INT, ENUM, STRUCT, UNION, DATASEC and ENUM64.
12           * "size" tells the size of the type it is describing.
13           *
14           * "type" is used by PTR, TYPEDEF, VOLATILE, CONST, RESTRICT,
15           * FUNC, FUNC_PROTO, VAR, DECL_TAG and TYPE_TAG.
16           * "type" is a type_id referring to another type.
17           */
18          union {
19                  __u32 size;
20                  __u32 type;
21          };
22  };
```

BTF provides useful metadata to debug eBPF programs, but another important use-case was found for the same information, which is not discussed.

While the eBPF sybsystem provides a stable sandbox for eBPF programs, an issue appears when we have programs that interact with the rest of the kernel's types and structures, such as programs that attach to kprobes and traceporints. The kernel's internal API is famously unstable, which allows for fast development, code refactoring and decreases technical debth over time. However, this decision makes it difficult to distribute external code that is loaded in the kernel; kernel modules have a strict requirement that they need to be compiled for the same kernel version as the kernel they are loaded with for this exact reason, which is unconvenient. Due to the popularity and usefullness of eBPF a new system was designed.

A first solution to this problem was attempted by the *BPF Compiler Collection* (BCC). The approac was to distribute C source code directly, then compiled on the target machine. This technically worked but it required the entire compilation toolchain (clang/LLVM) to be installed on the host machine. Since LLVM is considered an heavy dependency, a new solution had to be designed.

The new approach was called *Compile Once, Run Everywhere* (CO-RE) and is based on relocations, like the ones used when linking a program. The key idea is to mark memory accesses as relocatable items, which are resolved by the loader program using BTF information before loading the program. This can be performed by a self contained executable without requiring heavy dependencies. In order to resolve the relocations, the loader needs to know the BPF bytecode of the program, its relocation records, and the BTF information of all kernel types and symbols. This last data is exposed to userspace via the files `$SYSFS/kernel/btf/vmlinux` and `$SYSFS/kernel/btf/$MODULE` for dynamically loaded modules. With this approach, if the address of a member in a struct changes, then the relocation will take care of setting the right offset before the program is loaded, leveraging BTF information in order to make the eBPF program portable across kernel versions. The main steps of this operation are now described. Before loading a program, the loader reads the program's relocation records which the compiler generated in the `.BTF.ext` section. Then, it checks the current kernel's BTF information from the files in sysfs, and resolves the relocations using the BTF information. It finally loads the final progam via the bpf systemcall. The kernel is completely CO-RE agnostic since the relocations are done in userspace by libbpf.

4.8 Security Considerations

Since eBPF program are JIT compiled and executed as kernel code, several security implications need to be addressed. An external attacker may compromise an user with permissions to load eBPF program and modify maps, or it may compromise an eBPF program that is being loaded

by a privileged process. This section models these two scenarios, analyzes the impact on the CIA triad (confidentiality, integrity, availability), and explores some security hardening solutions aimed to mitigate these attacks and decrease risk. Several vulnerabilities on the implementation of several parts of the eBPF subsystem have been found [8] such as CVE-2016-4557 [6] and CVE-2021-3490 [7]. This section focuses on the security of the architecture instead of particular implementation bugs.

Now suppose that an attacker is able to load an arbitrary eBPF program or tamper with the execution of other eBPF programs by reading or writing to their maps. This could happen, for example, by compromising a privileged user or a privileged process that grants elevated permissions to the attacker. The abilities of such actor would be based on the type of program begin loaded, and are quite broad:

- *Confidentiality*: the verifier only ensures that an eBPF program of a certain type reads a limited set of well-defined resources, thereby restricting the scope of the attack. Still, eBPF program can pose a confidentiality issue. For example, an XDP program could read unencrypted information on the network, such as passwords or other data which may be confidential; a kprobe program could read values from registers and a tracepoint program could read an event's data, which may contain confidential information.

- *Integrity*: the verifier checks that an eBPF program only writes to where it is supposed to do, so a program cannot modify arbitrary memory. Yet, an XDP eBPF program can rewrite the contents of network packets.

- *Availability*: the verifier asserts that an eBPF program terminates, so that an eBPF program is not able to completely halt the system. However, programs can still cause meaningful availability disruption. For example, some types of programs can drop network packets, thereby making the host unreachable through the network. LSM programs can deny permissions to perform various types of operations to other users, such as blocking systemcalls. Intentionally slow and memory-heavy eBPF programs can slow down the system and take available memory.

This analysis shows that an attacker with high privileges can meaningfully tamper with the system and pose a security threat. Therefore, defenders should always follow the principle of least privileges by limiting the capabilities of an user to be as restrictive as possible. eBPF provides several solutions to harder the system and restrict usage, which are explained next.

Capabilities

A common security safeguard is to fully disable access to eBPF programs for unprivileged users by setting the kernel parameter `kernel.unprivileged_bpf_disabled` to true (1), which is the default in most Linux distributions. Users can get different levels of privileges through Linux *capabilities*, which are per-thread attributes used for fine grained access control: if unprivileged users are disabled, then only users with the `CAP_BPF` or `CAP_SYS_ADMIN` capabilities can load programs. Additional capabilities are needed to load specific eBPF programs; for example, `CAP_NET_ADMIN` is required to load networking programs, and `CAP_PERFMON` is necessary for tracing programs. This limits the operations that an user is able to perform, reducing risk.

eBPF tokens

eBPF supports delegating some functionalities from a privileged process to an unprivileged one thought a system of *tokens*. The unprivileged process will be able to perform only the operations that were specifically allowed. An example use-case would be for a container runtime that gives specific permissions to the container based on the workload's needs.

LSM policies

More sophisticated policies can be implemented though a Linux Security Module because of the many security hooks placed where the user interacts with the eBPF subsystem. An LSM policy can deny or allow certain operations. For example, a policy could block the loading of any eBPF program that tampers with network traffic, it could also allow only eBPF programs

that do read-only tracing, or implement additional access control checks to verify that the user is legitimate. This again limits or eliminates the possible misuse of eBPF.

eBPF Program signing

Suppose the attacker is able to control or modify the eBPF program that a privileged loader will load on the kernel. The loader may have the necessary permissions to load an eBPF program that interacts with the network, so an attacker may tamper the eBPF program to perform availability or confidentiality attacks.

To protect the operating system against these types of attacks, a technique called *program signing* was introduced. The idea is to allow only the loading of signed eBPF programs from an accepted key, similarly to how kernel modules are signed. This ensures that only programs released by trusted authors are loaded. Implementing this policy is not straight forward: due to CO-RE and other dynamic behaviour, eBPF programs are modified before being loaded by the loader to perform relocations and other necessary operations. Consequently, the program that is being distributed is not the same program that is loaded, and checking the signature only at load time would not be possible. This issue has been highly discussed and practical solutions are still being debated. The current proposal leverages a chain of trust between the kernel and the loader; the idea is based on a *trusted hash chain*: first, metadata of an eBPF program is hashed, then the code of the loader for that particular program is hashed and signed together with the hash of the eBPF program, finally the kernel will check that the hash of the loader is signed with a trusted key. This final signature check happens when loading the eBPF program, but before the LSM hook. This chain makes sure that a trusted loader is able to load only a trusted program and perform the necessary relocations. Other details are still evolving.

4.9 eBPF in production

eBPF is being leveraged in many contexts and by major institutions. For example, a Linux Foundation document [4] reported that S&P Global, Ikea, Sky, Alibaba and Red Hat use eBPF-based solutions for network optimization; Google, Netflix, Microsoft, Linkedin and others highly rely on eBPF for deep network observability; Shopify and Apple actively use eBPF for runtime security and policy enforcement.

Cloudflare used eBPF to quickly patch vulnerable Linux machines from a critical kernel exploit without rebooting their systems [2]. eBPF-based schedulers were shown to outperform the default Linux scheduler [1] and are being deployed for large production workloads at Meta [5].

5 Kivebpf

In this chapter, an example eBPF application used for kernel-level tracing is presented: *Kivebpf*.

5.1 Introduction

Kivebpf is a kubernetes-native, eBPF-powered file access monitoring solution. The user of the application is able to select files to monitor using match queries, such as file paths and filters on Kubernetes pods or containers; when one of the monitored files is accessed, such as when it is read from or written to, the application will generate an alert.

The application is implemented as a single Kubernetes operator and is structured into multiple components that interact with each other, Kubernetes or the operating systems. The code of the project is open source and available on GitHub [10], and the official documentation is also available online [9]. This chapter focuses on the eBPF program component of the application.

5.1.1 Resources

Resources are Kubernetes objects that are used to save state. They can be created or edited by the user and the application. When a resource is changed, the application will react to it and update itself if necessary via a mechanism called *reconciliation*. Kivebpf registers two resources: `KivePolicy` and `KiveData`.

A *KivePolicy* is created by the user of the application to place *traps* on certain files living on Kubernetes pods. The user can specify match rules to place the trap on all pods that match certain conditions.

A `KivePolicy` looks like this:

```
apiVersion: kivebpf.san7o.github.io/v1
kind: KivePolicy
metadata:
  labels:
    app.kubernetes.io/name: kivebpf
  name: kive-sample-policy
  namespace: kivebpf-system
spec:
  alertVersion: v1
  traps:
    - path: /secret.txt
      create: true
      mode: 444
      callback: "http://my-callback.com/alerts"
      matchAny:
      - pod: nginx-pod
        namespace: default
        ip: 192.168.0.3
        containerName: "regex: .*"
        matchLabels:
          security-level: high
      metadata:
          severity: critical
```

Where the "path" option specifies the path, which is going to be created if it does not exist when the "create" option is set to "true". The user can also specify a "callback" URL which

will be used to send alerts. The match rules are specified with "matchAny", such as pod names, namespaces and labels.

From a `KivePolicy`, the application generates various `KiveData` resources, one for each trap. This holds metadata and other information needed for the application to identify the file and generate the alert. A `KivePolicy` looks like the following:

```
apiVersion: kivebpf.san7o.github.io/v1
kind: KiveData
metadata:
  annotations:
    kive-alert-version: v1
    kive-policy-name: kive-sample-policy
    callback: ""
    container-id: containerd://da9e46ae1873ec463...
    container-name: nginx
    namespace: default
    node_name: kive-worker2
    path: /secret.txt
    pod_ip: 10.244.1.2
    pod_name: nginx-pod
  creationTimestamp: "2025-07-25T08:06:12Z"
  generation: 1
  name: kive-data-nginx-pod-default-13667586
  namespace: kivebpf-system
  resourceVersion: "23395"
  uid: 788bcb67-a9de-480d-a179-e40234116459
  labels:
      trap-id: c4705ec263cc353100b6f18a12...
      kernel-id: fc9a30d5-6140-4dd1-b8ef-c638f19ebd71
spec:
  inode-no: 13667586
  dev-id: 3
  metadata:
    severity: critical
```

5.1.2 Alert

When a monitored file is accessed, an *alert* is generated, which is referred to as *Kivealert*. The alert contains data in json format, providing various information about the event.

An example alert is the following:

```
{
  "kive-aler-version": "v1",
  "kive-policy-name": "kive-sample-policy",
  "timestamp": "2025-08-02T16:51:19Z",
  "metadata": {
    "path": "/secret.txt",
    "inode": 16256084,
    "mask": 36,
    "kernel-id": "2c147a95-23e5-4f99-a2de-67d5e9fdb502"
  },
  "custom-metadata": {
    "severity": "critical",
  },
  "pod": {
```

```
    "name": "nginx-pod",
    "namespace": "default",
    "container": {
      "id": "containerd://0c37512624823392d71e99...",
      "name": "nginx"
    }
  },
  "node": {
    "name": "kive-worker"
  },
  "process": {
    "pid": 176928,
    "tgid": 176928,
    "uid": 0,
    "gid": 0,
    "binary": "/usr/bin/cat",
    "cwd": "/",
    "arguments": "/secret.txt -"
  }
}
```


5.1.3 Components

The components used by the application are:

- *discover controller*: collects data from the files to monitor, such as inode number and device id. Manages the `KivePolicy` resource. There must be one discover controller for each node. This is necessary because the controller has to interface directly to the container runtime and access the filesystem of the container in order to read the inode. Therefore, each discover controller instance will react to all the changes in the `KivePolicy` resource, and generate one or multiple `KiveData` resource[s] for this specific kernel. Any discover controller[s] may generate multiple `KiveData` resources from the same `KivePolicy`. This is intended by design since the policy may match multiple pods hence the relationship between a policy and a `KiveData` is one to many.

- *loader controller*: loads and updates the eBPF program into the kernel. Generates alerts, called `KiveAlerts` and manages the `KiveData` resource. There must be one loader controller for each running kernel. This is necessary because the loader interacts directly with the running kernel. It is useless to have multiple loaders in the same kernel, but at least one is necessary to load the eBPF program. To implement this, each node needs to fetch its running kernel identifier and then run elections so that only one node is elected per running kernel.

- *eBPF program*: informs the loader that one of the monitored files was accessed, communication additional information from the kernel.

- *pod controller*: reacts to changes in Kubernetes pods to update the state of the application. Since Pods are global resources accessed from anywhere, we need only one pod controller in the cluster.

5.2 Detecting file accesses

The application uses an eBPF program to monitor file accesses. The eBPF program is executed when a certain kernel function is called through a kprobe; it will check if the probed function interacts with any of the monitored files specified by the user. If that is the case, then it logs the event with additional metadata, such as which process ID that called the function. The choice of what function to hook to is crucial for the correct behaviour of the application.

The information of which files to check is provided from userspace to the eBPF program via a map, which is a shared array between userspace and kernelspace and can be read from and written to by both actors.

The eBPF program needs to be loaded and updated when the user changes the monitoring policy (`KivePolicy`) or the cluster changes or updates its topology.

5.2.1 Uniquely identifying a file

To identify a particular file, a simple solution is to use its path. There cannot be two different files with the same path name, however symbolic links (*symlinks*) can alias a file thereby accessing its contents though a different path. To circumvent this, Kivebpf is using the inode number instead: each file has an inode number that is unique in the filesystem where it is saved in, is is used by the filesystem to internally identify data. Yet, there is still a case that needs to be taken into account: different filesystems may have different files with the same inode number. This happens because the inode number is an identifier in a particular filesystem, but it may identify a different file in another filesystem. To solve this problem, Kivebpf uses both the inode number and the device id, which is unique per filesystem, to generate an unique identifier. Even if a filesystem is binded onto another, the binded filesystem and the filesystem onto which the binded one is mounted onto have different inode numbers, hence the key remains unique.

The component that is responsible to fetch the inode number is the discover controller. The loader controller and the discover controller share information through a `KiveData` resource.

5.3 eBPF Program

To check whether an actor has interacted with a file, the eBPF program hooks to the function `inode_permission` through a keyprobe. This function gets called every time the permissions of an inode are checked in the kernel, which appends before any operation on files. This hook allows the eBPF program to log information whenever a permission is checked, accessing the permissions requested, as well as the process id that tried to check the permissions. The eBPF program uses BTF types information to enable compile-once run everywhere (CO-RE). It will log information only if the probed function is called on an inode present in an `KiveData` resource. The loader will fetch those inodes and send them to the eBPF program via a map that contains an array of inodes and devices ids. The traced information is sent to userspace via a ringbuffer accessible by the operator for logging purposes. These are then processed by the loader to generate a `KiveAlert`.

5.3.1 Implementation

Implementation code of the eBPF program:

`maps.h`:

```
#ifndef _HIVE_MAPS_H_
#define _HIVE_MAPS_H_

#include "vmlinux.h"
#include "log_data.h"
#include <bpf/bpf_helpers.h>

#define MAP_MAX_ENTRIES 1024

struct map_key {
  long unsigned int inode;
  dev_t dev;
};

struct {
  __uint(type, BPF_MAP_TYPE_HASH);
```

```
  __type(key, struct map_key);
  __type(value, u8);
  __uint(max_entries, MAP_MAX_ENTRIES);
} traced_inodes SEC(".maps");

struct {
  __uint(type, BPF_MAP_TYPE_RINGBUF);
  __uint(max_entries, 1 << 24);
  __type(value, struct log_data);
} rb SEC(".maps");

#endif // _HIVE_MAPS_H_
```

log_data.h:

```
#ifndef _HIVE_DATA_H_
#define _HIVE_DATA_H_

#include "vmlinux.h"

#ifndef TASK_COMM_LEN
#define TASK_COMM_LEN 16
#endif

struct log_data {
  pid_t pid;                /* process id */
  gid_t tgid;               /* thread group id */
  uid_t uid;                /* user id */
  gid_t gid;                /* group id */
  dev_t dev;                /* device id */
  long unsigned int ino;    /* inode number */
  int mask;                 /* Octal representation of file permissions */
  char comm[TASK_COMM_LEN]; /* name of the executable of the task */
};

#endif // _HIVE_DATA_H_
```

tracer.bpf.c:

```
#include "vmlinux.h"
#include "maps.h"
#include "log_data.h"
#include "license.h"

#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>
#include <bpf/bpf_core_read.h>

/*
 *  Fill and send struct log_data to the ring buffer.
 */
static __always_inline void
kprobe_output(long unsigned int inode, dev_t dev, int mask)
```

```
{
  struct log_data data = {};

  __u64 pid_tgid = bpf_get_current_pid_tgid();
  __u64 uid_gid = bpf_get_current_uid_gid();

  data.tgid = pid_tgid >> 32;
  data.pid = (gid_t) pid_tgid;
  data.gid = uid_gid >> 32;
  data.uid = (gid_t) uid_gid;
  data.ino = inode;
  data.dev = dev;
  data.mask = mask;
  bpf_get_current_comm(data.comm, TASK_COMM_LEN);

  bpf_ringbuf_output(&rb, &data, sizeof(struct log_data), 0);
}

/*
 *  Probed function:
 *  int inode_permission(struct mnt_idmap *idmap,
 *                       struct inode *inode, int mask)
 *  Description: Check if accessing an inode is allowed
 */
// kprobe_inode_permission_old should be loaded on kernels < 5.12
SEC("kprobe/inode_permission")
int kprobe_inode_permission_old(struct pt_regs *ctx)
{
  struct inode *inode = (struct inode*) PT_REGS_PARM1(ctx);
  int mask = (int) PT_REGS_PARM2(ctx);

  if (!inode)
    return 0;

  long unsigned int ino = BPF_CORE_READ(inode, i_ino);
  dev_t dev = BPF_CORE_READ(inode, i_sb, s_dev);
  struct map_key key = {};
  key.inode = ino;
  key.dev   = dev;

  if (bpf_map_lookup_elem(&traced_inodes, &key)) {
    kprobe_output(ino, dev, mask);
    return 0;
  }

  return 0;
}

// kprobe_inode_permission_new should be loaded on kernels >= 5.12
SEC("kprobe/inode_permission")
int kprobe_inode_permission_new(struct pt_regs *ctx)
{
  struct inode *inode = (struct inode*) PT_REGS_PARM2(ctx);
```

```
68    int mask = (int) PT_REGS_PARM3(ctx);
69
70    if (!inode)
71      return 0;
72
73    long unsigned int ino = BPF_CORE_READ(inode, i_ino);
74    dev_t dev = BPF_CORE_READ(inode, i_sb, s_dev);
75    struct map_key key = {};
76    key.inode = ino;
77    key.dev   = dev;
78
79    if (bpf_map_lookup_elem(&traced_inodes, &key)) {
80      kprobe_output(ino, dev, mask);
81      return 0;
82    }
83
84    return 0;
85  }
```

6 Conclusions

This work presented an analysis of Linux kernel tracing, with the objective of understanding the systems and interfaces involved in tracing kernel events, as well as the evolution of the mechanisms used to implement them. The study examined the main tracing frameworks available in Linux, with particular attention to ftrace and perf, and explored the events exposed by the kernel and the capabilities offered by these tools. It also considered the role of modern tracing applications, focusing on eBPF. The analysis provided an overview of the underlying mechanisms and their implementation, examining the relevant source code and the technical details necessary to understand their behaviour.

Linux tracing remains a continuously evolving field, reflecting the ongoing development of the kernel and the increasing demands placed on observability, performance analysis, and security. New features, changes to existing interfaces, and improvements to the underlying implementation require tracing tools and the knowledge surrounding them to evolve accordingly. For this reason, the results presented in this work should be considered within the context of a system that is continually changing, rather than as a definitive description of a fixed set of mechanisms.

It is hoped that this analysis will prove useful to both developers already working with Linux tracing and those approaching the subject for the first time, helping them navigate the relevant interfaces, understand the underlying mechanisms, and make informed decisions when developing or maintaining tracing tools. In this respect, the work aims to contribute to the knowledge required to follow the evolution of Linux tracing and to support further development in this area.

Bibliography

[1] Saman Akbari and Manfred Hauswirth. Towards high-performance ebpf-based os scheduling for serverless functions. In *Proceedings of the 17th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '26, page 8–14, New York, NY, USA, 2026. Association for Computing Machinery.

[2] Chris J. Arges, Sourov Zaman, and Rian Islam. How Cloudflare responded to the "Copy Fail" Linux vulnerability. Cloudflare Blog, May 2026. Accessed: 2026-10-18.

[3] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 238–252. ACM, 1977.

[4] Bill Doerrfeld. eBPF In Production Report. Technical report, The Linux Foundation / eBPF Foundation, February 2026. Accessed: 2026-10-18.

[5] Tejun Heo. sched: Implement BPF extensible scheduler class. LWN.net, May 2024. Accessed: 2026-10-18.

[6] MITRE. CVE-2016-4557 record. `https://www.cve.org/CVERecord?id=CVE-2016-4557`, 2016. Accessed: 2026-10-16.

[7] MITRE. CVE-2021-3490 record. `https://www.cve.org/CVERecord?id=CVE-2021-3490`, 2021. Accessed: 2026-10-16.

[8] Mohamed Husain Noor Mohamed, Xiaoguang Wang, and Binoy Ravindran. Understanding the security of linux ebpf subsystem. In *Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '23, page 87–92, New York, NY, USA, 2023. Association for Computing Machinery.

[9] San7o. kivebpf: Documentation. `https://san7o.github.io/kivebpf/index.html`, 2026. Accessed: 2026-10-21.

[10] San7o. kivebpf: Source code. `https://github.com/San7o/kivebpf`, 2026. Accessed: 2026-10-21.

[11] D. Thaler. BPF Instruction Set Architecture (ISA). RFC 9669, October 2024.

[12] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. Verifying the verifier: eBPF range analysis verification. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III*, pages 226–251, Berlin, Heidelberg, 2023. Springer-Verlag.